

GeneralEquilibriumModeling.jl and Structural Equilibrium Models

A Cookbook

LI Wu (liwu@staff.shu.edu.cn)

2026-09-10

Table of contents

Preface	5
1 The von Neumann Equilibrium Model and Structural Equilibrium Models	7
1.1 The Eigensystem Form of General Equilibrium Models	7
1.1.1 Eigenvalue Equations and the von Neumann Equilibrium Model	7
1.1.2 Structural Equilibrium Models	8
1.2 Complementarity Relations in the von Neumann Equilibrium Model	9
1.3 Mixed Complementarity Problems and Their Solution in Julia	10
1.3.1 Mixed Complementarity Formulation and Numerical Solution	10
1.3.2 General Equilibrium Modeling Frameworks	11
2 The GEMB High-Level Framework, the GEMB Low-Level Framework, and the GEM Framework	13
2.1 Basic Example: A 2×2 Cobb–Douglas Equilibrium with Production	13
2.2 GEMB High-Level Framework	15
2.3 GEMB Low-Level Framework	17
2.4 GEM Framework	20
2.4.1 Net Supply Representation	20
2.4.2 Economic Agents	22
2.4.3 Agent Conditions	23
2.4.4 Commodity Market Conditions	24
2.4.5 Auxiliary Variables and Auxiliary Equations	24
2.4.6 Equilibrium Model and Solution	25
2.5 Total Revenue–Expenditure Balance Conditions (TREBC) and Wide-Sense Equilibrium	27
2.6 Cost–Minimization Conditions and Marginal–Utility Conditions	30
2.7 Production Stationarity Conditions and Explicit Agent Conditions	32
2.8 Activity–Demand Specifications	34
2.9 Custom Net-Supply Agents and Condition Agents	36
2.9.1 Custom Net-Supply Agents and Multi-Activity Producers	36
2.9.2 Condition Agents	36
2.9.3 A Two-Commodity von Neumann Model with Three Activities	37
2.10 Equilibrium Results and Statistics	40
2.10.1 Equilibrium Statistics	40
2.10.2 Demand and Supply Matrices	41

2.10.3	Printing and Checking an Equilibrium	43
3	Static Pure-Exchange Equilibrium	44
3.1	CES and DCES Consumers: A 2×2 Pure-Exchange Model	44
3.1.1	CES and DCES Specifications	44
3.1.2	A 2×2 Pure-Exchange Model	46
3.2	Marshallian-Demand Consumers: A 2×2 Pure-Exchange Model	50
3.3	Multiple Equilibria: A 2×2 Model	53
3.4	Money Illusion and Absolute-Price-Dependent Utility: A 2×2 Model	56
3.5	Relative-Price-Dependent Utility: A 2×2 Model	60
3.6	Asset-Exchange Equilibrium with AMSD Utility: A 2×2 Model	64
3.7	Matthew Effect in Asset-Exchange Equilibrium: A 2×2 Model	66
4	Static Equilibrium with Production: Basic Models	69
4.1	Production and Transformation Specifications	69
4.1.1	Production-Function Specification	69
4.1.2	Power-Production Specification	70
4.1.3	CET Transformation Specification	72
4.2	Equivalent Representations Using Auxiliary Commodities and Virtual Producers	73
4.3	CES-CET Production: A 4×2 Model	74
4.4	Ad Valorem Tax and Ad Valorem Claim: A 3×2 Model	78
4.4.1	Ad Valorem Claim	78
4.4.2	A 3×2 Model with Ad Valorem Tax	78
4.5	Specific Tax: A 3×2 Model	82
4.6	Non-rival Service and Lindahl Prices: A 3×3 Model	85
4.7	Capital Rental and Equity: A 4×3 Model	88
5	Static Equilibrium with Production: Extended Models	91
5.1	Pollution and Negative Prices: A 3×2 Model	91
5.2	Producer-to-Consumer Externality: A 2×2 Model	94
5.3	Marginal Pricing Principle and Scale Claim Rates	96
5.3.1	Scale Claim Rates	96
5.3.2	Marginal Pricing with a Power Production Function: A 3×2 Model . .	97
5.3.3	Marginal Pricing with an Exponential Production Function: A 3×2 Model	100
5.4	Degree-One Homogenization of Production Functions	103
5.4.1	Degree-One Homogenization and the Scale Claim Rate	103
5.4.2	Equivalence under Degree-One Homogenization	104
5.4.3	Power and Exponential Production Functions	105
5.4.4	Square-Sum Production Function and Marginal Pricing Equilibria: A 4×2 Model	109
6	Intertemporal Equilibrium	113
6.1	Intertemporal Model Construction	113

6.2	A Finite-Horizon Intertemporal Equilibrium Model	115
6.3	Endogenous Interest Rates: A Four-Period 8×2 Pure-Exchange Model	117
6.3.1	Endogenous Interest Rates	117
6.3.2	A Four-Period 8×2 Pure-Exchange Model	118
6.4	A Basic Overlapping-Generations Trading Model	122
6.4.1	The Model	122
6.4.2	Computational Implementation	125
References		128

Preface

Warning

Work in Progress

This manuscript is currently a draft. If you spot any errors or typos, or have suggestions, please feel free to submit feedback via [GitHub Issues](#).

This book provides a concise introduction to applied general equilibrium modeling. It aims to explain, as clearly and directly as possible, the basic structure of general equilibrium models, methods for constructing such models, and their numerical solution. The book has the following main features:

1. The mathematical models presented in this book are primarily based on the structural equilibrium model (SEM) developed by the author (Li, 2019). The SEM extends the von Neumann equilibrium model and provides a matrix-based framework for describing economic agents and their activities, input-output relationships, and market equilibrium conditions in a unified manner. A structural equilibrium model can also be transformed into a complementarity problem. Compared with its complementarity formulation, the structural equilibrium representation is generally more intuitive and makes the economic structure of the model and its equilibrium conditions easier to understand.
2. The numerical models in this book are primarily constructed and solved using `GeneralEquilibriumModeling.jl`, a Julia package developed by the author. The package contains two main modules: `GEM`, which provides the core representation and solution framework for general equilibrium models, and `GEMB`, which provides more user-friendly, concise, and economically oriented modeling tools built on `GEM`. The `GEM` module uses `PATHSolver.jl` as its underlying solver for the complementarity problems associated with general equilibrium models.
3. Compared with `MPSGE.jl`, which emphasizes concise construction and solution of computable general equilibrium (CGE) models, and `JCGE.jl`, which is oriented toward modular CGE modeling, `GeneralEquilibriumModeling.jl` places greater emphasis on the direct correspondence between economic theory and numerical models. In particular, its `GEMB` module is well suited for learning and teaching general equilibrium modeling. It allows users to begin with the behavior of economic agents and equilibrium conditions, understand structural equilibrium models and their complementarity representations, and gradually construct applied general equilibrium models.

4. This book is organized largely as a handbook of examples. A sequence of models, progressing from simple to more complex settings, is used to introduce the construction, representation, numerical solution, and economic interpretation of general equilibrium models.
5. The book is intended as a concise overview. Its emphasis is on the basic ideas, model structures, and practical methods of general equilibrium modeling rather than on extensive literature reviews, detailed theoretical proofs, or systematic instruction in programming syntax. Some Julia examples in this book have been adapted from examples in the author's earlier R packages, `GE` and `CGE`.

1 The von Neumann Equilibrium Model and Structural Equilibrium Models

1.1 The Eigensystem Form of General Equilibrium Models

1.1.1 Eigenvalue Equations and the von Neumann Equilibrium Model

Applied general equilibrium models can be represented in a variety of mathematical forms. An eigensystem is one important representation. It reveals the dual structure between prices and activity levels; when the model takes the form of an eigenvalue inequality system, it can also characterize the associated complementarity relations.

For a square matrix $\mathbf{A}$, the corresponding left and right eigenvalue equations can be written as

$$\begin{aligned}\mathbf{p}^\top \mathbf{A} &= \rho \mathbf{p}^\top \\ \mathbf{A} \mathbf{z} &= \rho \mathbf{z}\end{aligned}$$

where $\mathbf{p}$ and $\mathbf{z}$ are the left and right eigenvectors, respectively, and ρ is the eigenvalue. Extending the eigenvalue equations for a square matrix to two $n \times m$ matrices, $\mathbf{A}$ and $\mathbf{B}$, and relaxing the equalities to inequalities gives the generalized eigenvalue inequality system

$$\begin{aligned}\mathbf{p}^\top \mathbf{A} &\geq \rho \mathbf{p}^\top \mathbf{B} \\ \mathbf{A} \mathbf{z} &\leq \rho \mathbf{B} \mathbf{z}\end{aligned}\tag{1.1}$$

The von Neumann general equilibrium model is precisely such an eigenvalue inequality system (von Neumann, 1945; Kemeny, Morgenstern, and Thompson, 1956). Here, $\mathbf{A}$ and $\mathbf{B}$ are the input coefficient matrix and output coefficient matrix, respectively, $\mathbf{p} \geq \mathbf{0}$ is the price vector, and $\mathbf{z} \geq \mathbf{0}$ is the activity-level vector. The first inequality states that the input value of each production activity is no less than its discounted output value, while the second states that the total input of each commodity does not exceed its discounted total output. The scalar ρ is the eigenvalue of the system and, economically, represents the discount factor corresponding to the equilibrium growth rate. If the equilibrium growth rate is γ , then

$$\rho = \frac{1}{1 + \gamma}$$

1.1.2 Structural Equilibrium Models

The (strict-sense) structural equilibrium model extends the input and output coefficient matrices in the von Neumann model to matrix-valued functions which usually involve consumers' utilities (Li, 2019, chapter 3):

$$\begin{aligned} \mathbf{p}^\top \mathbf{A}(\mathbf{p}, \mathbf{u}, \mathbf{z}) &\geq \rho \mathbf{p}^\top \mathbf{B}(\mathbf{p}, \mathbf{u}, \mathbf{z}) \\ \mathbf{A}(\mathbf{p}, \mathbf{u}, \mathbf{z}) \mathbf{z} &\leq \rho \mathbf{B}(\mathbf{p}, \mathbf{u}, \mathbf{z}) \mathbf{z} \end{aligned}$$

where $\mathbf{A}(\mathbf{p}, \mathbf{u}, \mathbf{z})$ and $\mathbf{B}(\mathbf{p}, \mathbf{u}, \mathbf{z})$ are the unit demand matrix and unit supply matrix, respectively, and $\mathbf{u}$ is the vector of consumers' utility levels. Each row of the two matrices corresponds to a commodity, and each column corresponds to an economic agent. The price vector remains on the left-hand side of the system and is associated with the unit revenue-expenditure balance conditions, whereas the activity-level vector remains on the right-hand side and is associated with the supply-demand balance conditions (i.e., market-clearing conditions). The structural equilibrium model therefore remains an eigensystem. The activity-level vector in a strict-sense structural equilibrium model typically consists of producers' activity levels and the numbers of consumers of different types.

For notational simplicity, when $\rho \neq 1$ in a structural equilibrium model, ρ can be incorporated into $\mathbf{B}(\cdot)$. In this case, $\mathbf{B}(\cdot)$ becomes the discount-adjusted unit supply matrix, and the structural equilibrium model can be written as

$$\begin{aligned} \mathbf{p}^\top \mathbf{A}(\cdot) &\geq \mathbf{p}^\top \mathbf{B}(\cdot) \\ \mathbf{A}(\cdot) \mathbf{z} &\leq \mathbf{B}(\cdot) \mathbf{z} \end{aligned}$$

Let $\text{Diag}(\mathbf{z})$ denote the diagonal matrix with $\mathbf{z}$ on its main diagonal. The demand and supply matrices generated by the unit demand and unit supply matrices are

$$\mathbf{D}(\cdot) = \mathbf{A}(\cdot) \text{Diag}(\mathbf{z}), \quad \mathbf{S}(\cdot) = \mathbf{B}(\cdot) \text{Diag}(\mathbf{z})$$

respectively. Thus, each column of $\mathbf{D}(\cdot)$ or $\mathbf{S}(\cdot)$ gives the demand or supply of the corresponding economic agent at its activity level. The structural equilibrium conditions imply $\mathbf{p}^\top \mathbf{D}(\cdot) = \mathbf{p}^\top \mathbf{S}(\cdot)$ and $\mathbf{D}(\cdot) \mathbf{1} \leq \mathbf{S}(\cdot) \mathbf{1}$. This leads directly to the mixed-form wide-sense structural equilibrium model

$$\begin{aligned}\mathbf{p}^\top \mathbf{D}(\cdot) &= \mathbf{p}^\top \mathbf{S}(\cdot) \\ \mathbf{D}(\cdot) \mathbf{1} &\leq \mathbf{S}(\cdot) \mathbf{1}\end{aligned}$$

The first condition represents total revenue-expenditure balance for each economic agent, while the second states that the demand for each commodity does not exceed its supply.

If there are no commodities in excess supply at equilibrium, so that supply equals demand for every commodity, the supply-demand inequality becomes an equality. This gives the equality-form wide-sense structural equilibrium model

$$\begin{aligned}\mathbf{p}^\top \mathbf{D}(\cdot) &= \mathbf{p}^\top \mathbf{S}(\cdot) \\ \mathbf{D}(\cdot) \mathbf{1} &= \mathbf{S}(\cdot) \mathbf{1}\end{aligned}$$

The first equation states that every economic agent satisfies total revenue-expenditure balance in the accounting sense, while the second states that supply equals demand for every commodity.

Thus, the equality-form wide-sense structural equilibrium model has the form of an eigenvalue equation system, whereas the mixed-form wide-sense structural equilibrium model has the form of an eigenvalue inequality system.

1.2 Complementarity Relations in the von Neumann Equilibrium Model

The von Neumann equilibrium model Equation 1.1 implies complementary slackness conditions (Kemeny, Morgenstern, and Thompson, 1956). Right-multiplying the first inequality by $\mathbf{z}$ and left-multiplying the second inequality by $\mathbf{p}^\top$ gives

$$\mathbf{p}^\top \mathbf{A} \mathbf{z} = \rho \mathbf{p}^\top \mathbf{B} \mathbf{z}$$

That is, the total input value equals the discounted total output value. The von Neumann equilibrium model therefore implies the following two sets of complementarity conditions:

$$\begin{aligned}\mathbf{0} &\leq \mathbf{A}^\top \mathbf{p} - \rho \mathbf{B}^\top \mathbf{p} \perp \mathbf{z} \geq \mathbf{0} \\ \mathbf{0} &\leq \rho \mathbf{B} \mathbf{z} - \mathbf{A} \mathbf{z} \perp \mathbf{p} \geq \mathbf{0}\end{aligned}$$

The first set describes the complementarity relation between production activity levels and unit-value losses. If the unit cost of an activity is strictly greater than its discounted unit revenue, that activity must be inactive. Conversely, if an activity operates at a positive activity level, its unit cost must equal its discounted unit revenue.

The second set describes the complementarity relation between commodity prices and excess supply. If a commodity is in excess supply, its equilibrium price must be zero. Conversely, if the equilibrium price of a commodity is positive, its supply must equal its demand.

The von Neumann equilibrium model therefore has two sets of complementarity relations. On the economic-agent side, each activity level is complementary to the corresponding unit value loss. On the commodity side, each commodity price is complementary to the corresponding excess supply.

1.3 Mixed Complementarity Problems and Their Solution in Julia

1.3.1 Mixed Complementarity Formulation and Numerical Solution

A mixed complementarity problem is defined by a mapping $\mathbf{F}$ and lower and upper bounds on the variable vector $\mathbf{x}$.

Given a function

$$\mathbf{F} : \mathbb{R}^N \rightarrow \mathbb{R}^N$$

and lower and upper bounds $\mathbf{l}$ and $\mathbf{u}$ for the variable vector $\mathbf{x}$, a mixed complementarity problem seeks a vector $\mathbf{x}$ satisfying $\mathbf{l} \leq \mathbf{x} \leq \mathbf{u}$ such that, for each $i = 1, \dots, N$,

$$\begin{cases} F_i(\mathbf{x}) \geq 0, & x_i = l_i \\ F_i(\mathbf{x}) = 0, & l_i < x_i < u_i \\ F_i(\mathbf{x}) \leq 0, & x_i = u_i \end{cases}$$

These conditions can be written compactly in vector form as

$$\mathbf{F}(\mathbf{x}) \perp \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}$$

When $l_i = 0$ and $u_i = +\infty$, this reduces to the usual nonnegative complementarity condition

$$0 \leq F_i(\mathbf{x}) \perp x_i \geq 0$$

When $l_i = -\infty$ and $u_i = +\infty$, x_i is a free variable, and the corresponding complementarity condition reduces to the equation

$$F_i(\mathbf{x}) = 0$$

A mixed complementarity problem can therefore represent, within a single model, nonnegative variables, variables with lower and upper bounds, free variables, equations, and complementarity conditions.

Complementarity problems have a standardized mathematical form and can be solved directly using specialized solvers such as PATH. They are therefore particularly suitable as the computational representation of general equilibrium models (Ferris and Munson, 2000). From an economic perspective, however, complementarity formulations are often less intuitive because economic relationships are decomposed into variables, residual functions, and their associated complementarity conditions.

By contrast, a structural equilibrium model represents the basic structure of an economic system directly in terms of demand matrices, supply matrices, revenue-expenditure balance conditions, and supply-demand balance conditions. This representation makes the economic relationships relatively transparent and facilitates theoretical analysis and the derivation of analytical solutions. A useful modeling strategy is therefore to construct and analyze an economic model first as a structural equilibrium model and then transform it into a complementarity problem for numerical solution.

In Julia, mixed complementarity models can be constructed with JuMP and solved using the PATH solver through `PATHSolver.jl`.

Because a general equilibrium model typically determines only relative prices, one commodity must normally be selected as the numeraire before solving the model with PATH, or an explicit price-normalization condition must be imposed. By Walras' law, one market-clearing condition is typically redundant. Thus, when the price of a commodity is fixed as the numeraire, the corresponding redundant market-clearing condition normally needs to be removed or replaced. In models with multiple equilibria, the particular equilibrium found by PATH may depend on the initial values. After solving the model, it is therefore advisable to check complementarity residuals, market-clearing errors, revenue-expenditure balance errors, and the solver termination status.

1.3.2 General Equilibrium Modeling Frameworks

`GeneralEquilibriumModeling.jl` is a Julia package for constructing and solving general equilibrium models. It emphasizes the correspondence between economic-theoretical models and their numerical solution forms. In the GEMB high-level framework, a model is specified in terms of commodities, economic agents, activity levels, revenue-expenditure balance conditions, and supply-demand balance conditions. In the GEMB low-level framework, these conditions are represented more directly at the computational level. The resulting equilibrium system is transformed into a mixed complementarity problem, represented numerically through JuMP, and solved with PATH through `PATHSolver.jl`.

`GeneralEquilibriumModeling.jl` provides three modeling frameworks.

1. The **GEM** framework is the lowest-level core representation and solution framework for general equilibrium models. Users can directly define economic agents' net supply functions, endogenous variables, and associated complementarity conditions, and may also introduce auxiliary variables and auxiliary equations. **GEM** constructs the resulting mixed complementarity problem through **JuMP** and solves it using **PATHSolver.jl**.
2. The **GEMB** low-level framework uses interfaces such as **CESSpec**, **ActivityDemandSpec**, and **build_agent** to transform specifications of production, consumption, and other economic behavior into **GEM.NetSupplyAgent** objects.
3. The **GEMB** high-level framework organizes commodities and economic agents through **GEMBModel**, commodity names or **CommodityRef**, and interfaces such as **add_agent!**. Users therefore normally do not need to work directly with commodity indices, net supply functions, or low-level complementarity conditions.

Models constructed at all three levels are ultimately transformed into **GEM** equilibrium models and solved through a common numerical solution interface.

GeneralEquilibriumModeling.jl, **MPSGE.jl**, and **JCGE.jl** can all be used to construct and solve general equilibrium models, but their primary orientations differ. **GeneralEquilibriumModeling.jl** takes economic agents, net supply relations, and equilibrium conditions as its basic modeling objects. It emphasizes the direct correspondence between economic theory and numerical models and is intended primarily for learning and teaching general equilibrium theory, while also supporting research on structural equilibrium models and nonstandard equilibrium mechanisms.

MPSGE.jl is oriented primarily toward CGE modeling. It emphasizes concise representations of calibrated economic structures through production sectors, commodities, consumers, and nested production and utility functions. **JCGE.jl**, by contrast, places greater emphasis on the complete workflow of applied CGE modeling, using modular blocks and standardized run specifications to support model assembly, data calibration, scenario simulation, validation, and comparison of results.

The **GE** package for R solves general equilibrium models more directly on the basis of structural equilibrium models and structural dynamic models. It simulates the period-by-period adjustment of prices, activity levels, and resource allocation, allowing the economic system to approach an equilibrium state gradually. Compared with solving a mixed complementarity problem directly with **PATH**, this iterative approach is generally less computationally efficient, but it makes the dynamic process through which an economy moves from disequilibrium toward equilibrium observable. It can therefore be useful for studying the mechanism through which equilibrium is formed.

2 The GEMB High-Level Framework, the GEMB Low-Level Framework, and the GEM Framework

`GeneralEquilibriumModeling.jl` provides three interconnected modeling frameworks: the GEM framework, the GEMB low-level framework, and the GEMB high-level framework. The GEM framework allows users to construct economic agents' net supply functions and their associated complementarity conditions directly. The GEMB low-level framework generates GEM economic agents through behavioral specifications and agent builders. The GEMB high-level framework organizes complete equilibrium models using commodity names, economic agent names, and high-level model interfaces. In most cases, the GEMB high-level framework is sufficient, and it is therefore the framework on which this book primarily focuses.

This chapter introduces the basic modeling methods of `GeneralEquilibriumModeling.jl` through some simple equilibrium examples.

2.1 Basic Example: A 2×2 Cobb–Douglas Equilibrium with Production

Assume the following:

1. The economy contains two commodities, product and labor, and two economic agents, a firm and a consumer. Product is taken as the numeraire commodity.
2. The firm uses product and labor to produce product. Its production function is $y = 2x_1^{1/2}x_2^{1/2}$, where x_1 and x_2 denote the inputs of product and labor, respectively, and y denotes total product output.
3. The consumer is endowed with $\omega > 0$ units of labor. The utility function is $u = x_1^{1/2}x_2^{1/2}$, where x_1 and x_2 denote the quantities of product and labor consumed by the consumer, respectively.

Let the price vector be $\mathbf{p} = (1, p_2)^\top$, the firm's activity level be y , and the consumer's activity level be the utility level u . The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, u, y) = \begin{bmatrix} \frac{y}{2} \sqrt{p_2} & u \sqrt{p_2} \\ \frac{y}{2\sqrt{p_2}} & \frac{u}{\sqrt{p_2}} \end{bmatrix}$$

and

$$\mathbf{S}(y) = \begin{bmatrix} y & 0 \\ 0 & \omega \end{bmatrix}$$

Consider the following equality-form wide-sense structural equilibrium model:

$$\begin{aligned} \mathbf{p}^\top \mathbf{D}(\mathbf{p}, u, y) &= \mathbf{p}^\top \mathbf{S}(y) \\ \mathbf{D}(\mathbf{p}, u, y) \mathbf{1} &= \mathbf{S}(y) \mathbf{1} \end{aligned}$$

The equilibrium price vector, firm activity level, and consumer utility level are, respectively,

$$\mathbf{p}^* = (1, 1)^\top, \quad y^* = \omega, \quad u^* = \frac{\omega}{2}$$

The equilibrium demand and supply matrices are

$$\mathbf{D}^* = \begin{bmatrix} \frac{\omega}{2} & \frac{\omega}{2} \\ \frac{\omega}{2} & \frac{\omega}{2} \end{bmatrix}, \quad \mathbf{S}^* = \begin{bmatrix} \omega & 0 \\ 0 & \omega \end{bmatrix}$$

In particular, when $\omega = 100$,

$$\mathbf{p}^* = (1, 1)^\top, \quad y^* = 100, \quad u^* = 50$$

In the next few sections, we will construct the basic example above using the GEMB high-level framework, the GEMB low-level framework, and the GEM framework, respectively.

2.2 GEMB High-Level Framework

The GEMB high-level framework is built on the GEMB low-level framework and the GEM framework. Users specify a model using commodity names, behavioral specifications, and declarations of economic agents. They do not need to assign commodity indices manually or directly construct `GEM.NetSupplyAgent` or `GEM.NetSupplyEquilibriumModel` objects. GEMB progressively converts the high-level model into a GEM equilibrium model and then calls the GEM solver.

The GEMB high-level framework mainly involves the following steps:

1. Use `GEMBModel` to specify the set of commodities, the numeraire commodity, and price bounds.
2. Use `CESSpec`, `DCESSpec`, or other behavioral specifications to describe production and consumption behavior.
3. Use `add_agent!` to add firms, consumers, and other economic agents.
4. Use `solve` to convert the high-level model into a GEM equilibrium model and solve it.

In the GEMB low-level framework, commodities are represented by integer indices. The GEMB high-level framework allows commodities to be referred to directly by name. GEMB automatically converts commodity names into the integer indices required by the underlying model.

During the solution process, GEMB sequentially performs commodity-name resolution, agent construction, aggregation of net supplies, and assembly of the GEM equilibrium model, and finally calls the solver provided through GEM. The solution is returned using the common equilibrium result type, from which information such as prices, activity levels, agents' net supplies, and market residuals can be obtained.

The main feature of the GEMB high-level framework is that it organizes models using economically meaningful commodity names and agent declarations. It hides technical details such as commodity indices, `NetSupplyAgent` construction, and low-level equilibrium-model assembly. For problems that require custom net supply functions or specialized complementarity conditions, users can instead work with the GEMB low-level framework or the GEM framework.

The GEMB low-level framework provides reusable construction of economic agents, whereas the GEMB high-level framework treats `GEMBModel` as a one-shot model representation. For each economic agent, the high-level model retains both the original economic specification and its resolved commodity mappings, together with the constructed net-supply agent used for equilibrium computation.

Listing 2.1 A 2×2 Cobb–Douglas Equilibrium Model: The GEMB High-Level Framework

```
# GEMB high-level framework: use commodity names and behavioral specifications.
using GeneralEquilibriumModeling.GEMB

alpha = 2.0
omega = 100.0
beta = [0.5, 0.5]

model = GEMBModel(
    [:product, :labor];
    numeraire=:product,
)

add_agent!(
    model,
    CESSpec(beta; es=1.0, alpha=alpha);
    outputs=:product,
    demands=[:product, :labor],
    activity_start=100.0,
    name=:firm,
)

add_agent!(
    model,
    CESSpec(beta; es=1.0, alpha=1.0);
    demands=[:product, :labor],
    endowments=:labor,
    endowment_quantities=omega,
    activity_start=100.0,
    name=:consumer,
)

result = solve(
    model;
    p0=[1.0, 2.0],
    residual_tol=1.0e-8,
    silent=true,
)

stats = equilibrium_statistics(model, result)

output = stats.activity_levels[1]
utility = stats.activity_levels[2]

flows = demand_supply_matrices(model, result)

D = flows.demand
S = flows.supply
Sbar = flows.net_supply

DV = flows.demand_value
SV = flows.supply_value
```


2.3 GEMB Low-Level Framework

The GEMB low-level framework lies between the GEM framework and the GEMB high-level framework. The GEM framework requires users to define economic agents' net supply functions and equilibrium conditions directly. The GEMB low-level framework instead allows users to first specify production, consumption, and other economic behaviors and then use `build_agent` to convert these behavioral specifications into `GEM.NetSupplyAgent` objects.

Compared with the GEMB high-level framework, however, the GEMB low-level framework still requires users to work relatively close to the underlying GEM representation. In particular, input commodities, output commodities, and endowments are generally specified by commodity indices, and the resulting agents are assembled and solved using the GEM framework.

The central idea of the GEMB low-level framework is to separate economic modeling into two steps: **behavioral specification** and **agent construction**.

A behavioral specification describes how an economic agent determines its demand as prices and its activity level change. For example, `ActivityDemandSpec` allows the user to provide a demand function directly. Common specifications such as `CESSpec` and `DCESSpec` provide more specialized representations for CES, Cobb–Douglas, and Leontief demand structures.

After the behavioral specification has been defined, `build_agent` converts it into a GEM economic agent. During this step, the user specifies such information as the relevant commodity indices, outputs, endowments, activity variable, initial value, and the condition rule associated with the agent.

For a firm, the activity variable usually represents its production level. Given prices and an activity level, the behavioral specification determines the firm's input demand. The firm's net supply is then obtained by subtracting its input demand from its output supply. Under constant returns to scale, input demand changes proportionally with the production activity level.

GEMB can associate the firm's activity level with a unit revenue-expenditure balance condition. Economically, this condition compares the expenditure required for one unit of production activity with the revenue generated by that activity. If unit expenditure exceeds unit revenue, the firm cannot operate at a positive activity level. If the firm operates at a positive activity level, unit expenditure and unit revenue must be equal.

For a consumer, the activity variable usually represents utility. Given prices and a utility level, the behavioral specification determines the compensated demand required to attain that utility level. The consumer's net supply is obtained by subtracting this consumption demand from the consumer's endowment.

The consumer's utility level can be associated with a total revenue-expenditure balance condition. This condition compares total consumption expenditure with the income generated by

the consumer's endowment. When the consumer attains a positive utility level, total expenditure equals endowment income.

The basic modeling procedure of the GEMB low-level framework is as follows:

1. Define the economic agent's behavioral specification using a type such as `ActivityDemandSpec`, `CESSpec`, or `DCESSpec`.
2. Use `build_agent` to specify the agent's input commodities, output commodities, endowments, activity variable, initial value, and other relevant settings, and convert the behavioral specification into a `GEM.NetSupplyAgent`.
3. Use `GEM.NetSupplyEquilibriumModel` to assemble the resulting economic agents and commodity markets, and then call a GEM solver to solve the equilibrium model.

In the GEM framework and the GEMB low-level framework, the user may explicitly map commodity names to commodity indices before passing them to `NetSupplyAgent` or `build_agent`, respectively. In the GEMB high-level framework, `GEMBModel` performs this mapping automatically and also assembles the resulting agents into the equilibrium model.

The GEMB low-level framework therefore retains relatively direct control over commodity indices, economic agents, activity variables, and equilibrium conditions, while reducing the need to write net supply functions manually. It is useful when users want more control than the GEMB high-level framework provides but do not want to construct every net supply function and complementarity condition directly in GEM.

Listing 2.2 A 2×2 Cobb–Douglas Equilibrium Model: The GEMB Low-Level Framework

```
# GEMB low-level framework: build NetSupplyAgents from ActivityDemandSpec.
using GeneralEquilibriumModeling

alpha = 2.0
omega = 100.0
beta = [0.5, 0.5]

c = (product=1, labor=2)

firm = GEMB.build_agent(
    GEMB.ActivityDemandSpec(
        (y, p) -> GEMB.CES_input(beta, y, p; es=1.0, alpha=alpha),
    );
    output_indices=[c.product],
    output_coefficients=[1.0],
    demand_indices=[c.product, c.labor],
    activity_start=100.0,
    name=:firm,
)

consumer = GEMB.build_agent(
    GEMB.ActivityDemandSpec(
        (u, p) -> GEMB.CES_input(beta, u, p; es=1.0, alpha=1.0),
    );
    demand_indices=[c.product, c.labor],
    endowment_indices=[c.labor],
    endowment_quantities=[omega],
    activity_start=100.0,
    name=:consumer,
)

model = GEM.NetSupplyEquilibriumModel(
    GEM.AbstractNetSupplyAgent[firm, consumer],
    [:product, :labor];
    numeraire_index=c.product,
    numeraire_value=1.0,
    price_lower_bounds=[1.0e-10, 1.0e-10],
)

result = GEM.solve_equilibrium_model_mcp_jump(
    model;
    p0=[1.0, 2.0],
    residual_tol=1.0e-8,
    silent=true,
)

output = result.agent_variable_values[1][1]
utility = result.agent_variable_values[2][1]

println("Prices = ", result.prices)
println("Output = ", output)
```

2.4 GEM Framework

The GEM framework is the foundational modeling and solution framework of `GeneralEquilibriumModeling.jl`. It takes the net supply functions of economic agents as its basic modeling objects and transforms agent behavioral conditions, commodity market-clearing conditions, and other auxiliary conditions into a unified mixed complementarity problem. The resulting problem is constructed with JuMP and solved numerically using PATHSolver.jl. Models constructed through the GEMB low-level framework and the GEMB high-level framework are also ultimately converted into GEM equilibrium models.

2.4.1 Net Supply Representation

In a wide-sense structural equilibrium model, each row of the demand matrix and the supply matrix corresponds to a commodity, and each column corresponds to an economic agent. Let the demand matrix and the supply matrix be denoted by $\mathbf{D}(\cdot)$ and $\mathbf{S}(\cdot)$, respectively. The **net supply matrix** is defined as

$$\bar{\mathbf{S}}(\cdot) := \mathbf{S}(\cdot) - \mathbf{D}(\cdot)$$

The element $\bar{S}_{ij}$ represents the net supply of commodity i by economic agent j . A positive value represents net supply, whereas a negative value represents net demand. The net supply matrix therefore combines the demand matrix and the supply matrix into a single representation.

Let $\mathbf{D}_{\cdot j}$ and $\mathbf{S}_{\cdot j}$ denote the demand vector and supply vector of economic agent j , respectively. Column j of the net supply matrix is the net supply vector of economic agent j :

$$\bar{\mathbf{s}}_j := \mathbf{S}_{\cdot j} - \mathbf{D}_{\cdot j}$$

Thus,

$$\bar{\mathbf{S}} = [\bar{\mathbf{s}}_1 \quad \bar{\mathbf{s}}_2 \quad \cdots \quad \bar{\mathbf{s}}_m]$$

where $\bar{\mathbf{s}}_j$ describes the net supplies of all commodities by economic agent j .

Using the net supply matrix, the mixed-form wide-sense structural equilibrium model can be written compactly as

$$\mathbf{p}^\top \bar{\mathbf{S}}(\cdot) = \mathbf{0}$$

$$\bar{\mathbf{S}}(\cdot) \mathbf{1} \geq \mathbf{0}$$

The first condition states that every economic agent satisfies revenue-expenditure balance. Since column j of $\bar{\mathbf{S}}$ is $\bar{\mathbf{s}}_j$, it is equivalent to

$$\mathbf{p}^\top \bar{\mathbf{s}}_j = 0, \quad j = 1, \dots, m$$

The second condition states that the total net supply of every commodity is nonnegative, or equivalently, that total supply is no less than total demand.

The total net supply vector of all economic agents is

$$\bar{\mathbf{s}} := \sum_{j=1}^m \bar{\mathbf{s}}_j = \bar{\mathbf{S}}\mathbf{1}$$

so the commodity-side condition can also be written as

$$\bar{\mathbf{s}} \geq \mathbf{0}$$

When free commodities and excess supply are allowed, the commodity market condition can be expressed more generally as

$$\mathbf{0} \leq \bar{\mathbf{S}}(\cdot)\mathbf{1} \perp \mathbf{p} \geq \mathbf{0}$$

or equivalently,

$$\mathbf{0} \leq \bar{\mathbf{s}} \perp \mathbf{p} \geq \mathbf{0}$$

That is,

$$\bar{\mathbf{s}} \geq \mathbf{0}, \quad \mathbf{p} \geq \mathbf{0}, \quad \mathbf{p}^\top \bar{\mathbf{s}} = 0$$

If the price of commodity i is positive, its market must clear. If commodity i has positive excess supply, its equilibrium price must be zero. When all equilibrium commodity prices are positive, the market condition reduces to

$$\bar{\mathbf{s}} = \mathbf{0}$$

For an economic agent with a constant-returns-to-scale production structure, let z_j denote its activity level and let $\bar{\mathbf{s}}_j^u$ denote its net supply vector per unit of activity. Then

$$\bar{\mathbf{s}}_j = z_j \bar{\mathbf{s}}_j^u$$

When $z_j > 0$, the unit revenue-expenditure balance condition is

$$\mathbf{p}^\top \bar{\mathbf{s}}_j^u = 0$$

If the economic agent is allowed to be inactive, the more general unit revenue-expenditure balance condition is

$$0 \leq -\mathbf{p}^\top \bar{\mathbf{s}}_j^u \perp z_j \geq 0$$

where $-\mathbf{p}^\top \bar{\mathbf{s}}_j^u$ is expenditure minus revenue per unit of activity, that is, the unit loss or unit excess cost.

The net supply representation therefore does not change the economic meaning of the wide-sense structural equilibrium model. It simply combines demand and supply into a single matrix. Because the GEM framework directly computes the net supply of each economic agent, it generally does not need to construct the complete demand matrix and supply matrix explicitly.

2.4.2 Economic Agents

The GEM framework uses **NetSupplyAgent** to represent an economic agent. Each economic agent mainly contains the following information:

1. The commodities associated with the agent.
2. The agent's endogenous variables, together with their lower bounds, upper bounds, and initial values.
3. A function that calculates net supply from the agent variables, commodity prices, and observed variables.
4. Complementarity conditions paired with the agent variables.
5. Names used to identify the agent and its variables.

Let $\mathbf{v}_j$ denote the local variable vector of economic agent j , $\mathbf{p}_j$ the local price vector for the commodities associated with that agent, and $\mathbf{o}_j$ the vector of observed variables. The agent's net supply function can then be written generally as

$$\bar{\mathbf{s}}_j = \bar{\mathbf{s}}_j(\mathbf{v}_j, \mathbf{p}_j, \mathbf{o}_j)$$

Local variables belong only to the economic agent itself, whereas observed variables may refer to variables of other agents, commodity prices, or auxiliary variables. This mechanism makes it possible to represent inter-agent relationships, endogenous tax rates, endogenous interest rates, and other economic relationships that must be determined jointly within the equilibrium model.

NetSupplyAgent is the unified agent representation used by the GEM framework. Firms, consumers, and condition agents may have different economic interpretations, but numerically they can all be represented as combinations of local variables, net supply functions, and local complementarity conditions.

2.4.3 Agent Conditions

For economic agent j , let the condition function paired with the local variable vector $\mathbf{v}_j$ be

$$\mathbf{c}_j = \mathbf{c}_j(\mathbf{v}_j, \mathbf{p}_j, \bar{\mathbf{s}}_j, \mathbf{o}_j)$$

If the lower and upper bounds of variable v_{jk} are $\underline{v}_{jk}$ and $\bar{v}_{jk}$, respectively, the corresponding mixed complementarity condition is

$$c_{jk} \perp \underline{v}_{jk} \leq v_{jk} \leq \bar{v}_{jk}$$

which means

$$\begin{cases} c_{jk} \geq 0, & v_{jk} = \underline{v}_{jk} \\ c_{jk} = 0, & \underline{v}_{jk} < v_{jk} < \bar{v}_{jk} \\ c_{jk} \leq 0, & v_{jk} = \bar{v}_{jk} \end{cases}$$

The GEM framework provides several commonly used condition rules:

1. **UnitRevenueExpenditureBalanceConditions** pairs unit loss with the agent's activity variable, which typically represents a production activity level. It is mainly applicable to constant-returns-to-scale activities for which unit net supply can be defined.
2. **TotalRevenueExpenditureBalanceConditions** pairs total loss with the agent's activity variable, which may, for example, represent a consumer's utility level.
3. **MarginalUtilityConsumerConditions** constructs the KKT conditions for a consumer's utility-maximization problem using marginal utilities, commodity prices, and a budget multiplier.

4. **ProductionStationarityConditions** constructs stationary production conditions from a production function and marginal product functions. A stationary point obtained from these conditions is not necessarily a cost-minimizing point.
5. **CostMinimizationKKTConditions** constructs the KKT conditions for a firm's cost-minimization problem.
6. **ExplicitAgentConditions** allows the user to define an agent's condition function directly when the required relationship cannot be represented by one of the existing condition rules.

The GEM framework therefore supports both standard condition rules and user-defined complementarity conditions.

2.4.4 Commodity Market Conditions

The GEM framework maps the local net supply vectors of all economic agents into a common commodity space and then aggregates them. The total net supply of commodity i is

$$\bar{s}_i = \sum_{j=1}^m \bar{s}_{ij}$$

When the lower bound of the commodity price is zero and there is no finite upper bound, the commodity market condition is

$$0 \leq \bar{s}_i \perp p_i \geq 0$$

If $p_i > 0$, then $\bar{s}_i = 0$, so the market for commodity i clears. If $\bar{s}_i > 0$, then $p_i = 0$, so a commodity in excess supply may become a free commodity.

The GEM framework also allows other lower and upper bounds to be imposed on prices. Once a numeraire commodity is selected, its price is fixed at a specified value, thereby eliminating the proportional indeterminacy of the general equilibrium price vector.

2.4.5 Auxiliary Variables and Auxiliary Equations

In addition to economic agent variables and commodity prices, the GEM framework allows auxiliary variables and auxiliary equations to be introduced through **AuxiliaryVariable** and **AuxiliaryEquation**.

Let q_k denote an auxiliary variable. A corresponding auxiliary equation may be written as

$$g_k(\mathbf{p}, \mathbf{v}, \mathbf{q}) = 0$$

Auxiliary variables can be used to represent exogenous price conditions, activity-level normalization conditions, and other model-closure relationships. Economic agents may observe auxiliary variables through `AuxiliaryVariableRef`, variables of other agents through `AgentVariableRef`, and commodity prices through `PriceVariableRef`.

2.4.6 Equilibrium Model and Solution

After the economic agents have been constructed, they can be combined into a complete equilibrium model using `NetSupplyEquilibriumModel`. The model mainly contains the set of economic agents, the set of commodities, price bounds, the numeraire, and optional auxiliary variables and auxiliary equations.

The GEM framework combines the following components into a single mixed complementarity problem:

1. The local complementarity conditions of the economic agents.
2. The market-clearing conditions for the commodities.
3. The auxiliary equations associated with auxiliary variables.
4. The lower and upper bound conditions for prices and other equilibrium variables.

The model can be solved using `solve_equilibrium_model_mcp_jump`. This function constructs the mixed complementarity problem with JuMP and calls `PATHSolver.jl` to obtain a numerical solution.

The solution is returned as an `EquilibriumResult`, which contains information such as equilibrium prices, agent variable values, agent net supplies, total net supply, auxiliary variable values, and complementarity residuals.

Listing 2.3 A 2×2 Cobb–Douglas Equilibrium Model: The GEM Framework

```
# GEM framework: define net supplies and condition rules directly.
using GeneralEquilibriumModeling.GEM

alpha = 2.0
omega = 100.0

c = (product=1, labor=2)

firm = GEM.NetSupplyAgent(
    [c.product, c.labor],
    (v, p) -> begin
        y = v[1]
        p_product, p_labor = p
        r = sqrt(p_labor / p_product)
        x = (y / alpha) .* [r, 1 / r]
        [y - x[1], -x[2]]
    end;
    variable_names=[:output],
    variable_start=[100.0],
    condition_rule=GEM.UnitRevenueExpenditureBalanceConditions(),
    name=:firm,
)

consumer = GEM.NetSupplyAgent(
    [c.product, c.labor],
    (v, p) -> begin
        u = v[1]
        p_product, p_labor = p
        r = sqrt(p_labor / p_product)
        x = u .* [r, 1 / r]
        [-x[1], omega - x[2]]
    end;
    variable_names=[:utility],
    variable_start=[100.0],
    condition_rule=GEM.TotalRevenueExpenditureBalanceConditions(),
    name=:consumer,
)

model = GEM.NetSupplyEquilibriumModel(
    [firm, consumer],
    [:product, :labor];
    numeraire_index=c.product,
    numeraire_value=1.0,
    price_lower_bounds=[1.0e-10, 1.0e-10],
)

result = GEM.solve_equilibrium_model_mcp_jump(
    model;
    p0=[1.0, 2.0],
    residual_tol=1.0e-8,
    silent=true,
```

2.5 Total Revenue-Expenditure Balance Conditions (TREBC) and Wide-Sense Equilibrium

In a strict-sense structural equilibrium model with constant returns to scale, a firm's unit loss and activity level satisfy a complementarity relation. This condition rules out the possibility that a firm remains inactive when a profitable production opportunity exists.

In GEM, `TotalRevenueExpenditureBalanceConditions` constructs the relevant condition using the firm's total loss rather than its unit loss. As a result, `TotalRevenueExpenditureBalanceConditions` permits a broader notion of equilibrium: a firm with constant returns to scale may remain inactive even when a positive-profit opportunity exists.

This point can be illustrated by a slight extension of the basic example. Suppose the economy contains two commodities, product and labor, with product as the numeraire commodity. There are two firms, both using Cobb–Douglas production technologies with input shares $(0.5, 0.5)^\top$.

Firm 1 has efficiency parameter $\alpha_1 = 2$, while firm 2 has efficiency parameter $\alpha_2 = 1$. The consumer is endowed with $\omega = 100$ units of labor.

If the usual unit-loss condition is used, firm 1 is more efficient. Therefore, at equilibrium, firm 1 produces while firm 2 remains inactive. This equilibrium can be regarded as a strict-sense equilibrium because no inactive firm has an unexploited positive-profit opportunity.

If `TotalRevenueExpenditureBalanceConditions` is used instead, then, in addition to the strict-sense equilibrium above, there is another equilibrium:

$$\mathbf{p} = \left(1, \frac{1}{4}\right)^\top, \quad z_1 = 0, \quad z_2 = 25, \quad u = 25$$

At this equilibrium, firm 1 could earn a positive profit if it entered production. However, because its activity level is zero, its total loss is also zero. `TotalRevenueExpenditureBalanceConditions` therefore allows firm 1 to remain inactive.

This equilibrium satisfies the total revenue-expenditure balance conditions and the market-clearing conditions, but it does not satisfy the unit revenue-expenditure balance conditions of the strict-sense structural equilibrium model. It can therefore be called a **wide-sense equilibrium**, whereas an equilibrium satisfying the conditions of the strict-sense structural equilibrium model can be called a **strict-sense equilibrium**.

Because wide-sense equilibria may involve multiple solutions and degenerate cases, the solution returned by PATH may be relatively sensitive to the initial values.

For some production technologies, particularly nonconvex technologies, the firm's net supply cannot be represented naturally as an activity level multiplied by an activity-independent unit net-supply vector. In such cases, unit revenue-expenditure balance conditions may no longer

provide an appropriate representation, whereas total revenue-expenditure balance conditions can be imposed directly on the firm's total revenue and total expenditure at its current activity level.

The program for solving the wide-sense equilibrium described above is as follows.

Listing 2.4 A 2×3 Cobb–Douglas Equilibrium Model: Total Revenue–Expenditure Balance Conditions

```
# GEMB high-level framework: total revenue-expenditure balance conditions.
using GeneralEquilibriumModeling

alpha = 2.0
omega = 100.0
beta = [0.5, 0.5]

model = GEMB.GEMBModel(
    [:product, :labor];
    numeraire=:product,
    numeraire_value=1.0,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec(beta; es=1.0, alpha=alpha);
    outputs=:product,
    demands=[:product, :labor],
    activity_start=0.0,
    condition_rule=GEM.TotalRevenueExpenditureBalanceConditions(),
    name=:firm1,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec(beta; es=1.0, alpha=1.0);
    demands=[:product, :labor],
    endowments=:labor,
    endowment_quantities=omega,
    name=:consumer,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec(beta; es=1.0, alpha=1.0);
    outputs=:product,
    demands=[:product, :labor],
    activity_start=1.0,
    condition_rule=GEM.TotalRevenueExpenditureBalanceConditions(),
    name=:firm2,
)

result = GEMB.solve(
    model;
    p0=[1.0, 0.27],
    residual_tol=1.0e-8,
    silent=true,
)

stats = GEMB.equilibrium_statistics(model, result)
```

2.6 Cost-Minimization Conditions and Marginal-Utility Conditions

A general equilibrium model does not have to rely only on demand functions derived in advance. The first-order conditions of firms' and consumers' optimization problems can also be incorporated directly into the equilibrium system. GEM provides `CostMinimizationKKTConditions` and `MarginalUtilityConsumerConditions` for constructing the complementarity conditions associated with firms' cost-minimization problems and consumers' utility-maximization problems, respectively. This approach does not require input demand functions or consumer demand functions to be derived beforehand, and it can naturally accommodate corner solutions in which some input or consumption quantities are zero.

In GEM, `CostMinimizationKKTConditions` pairs the firm's activity level with its total loss. It therefore incorporates three types of relations: the firm's revenue-expenditure balance condition, the cost-minimization conditions for individual inputs, and the production constraint. The corresponding endogenous variables are, respectively, the activity level, the quantities of the various inputs, and the multiplier associated with the production constraint.

It should be noted that KKT conditions are, in general, first-order necessary conditions for a local optimum. For cost-minimization problems with the usual appropriate convexity properties, they can characterize a cost-minimizing solution. For nonconvex production technologies, however, satisfying the KKT conditions alone does not necessarily imply that a global cost minimum has been obtained.

`MarginalUtilityConsumerConditions` constructs utility-maximization conditions from consumers' marginal utilities. It does not require the demand functions to be derived in advance. Instead, the consumption quantity of each commodity is paired directly with its corresponding marginal-utility condition, while the budget multiplier is paired with the budget constraint.

The following basic example uses cost-minimization conditions and marginal-utility conditions.

Listing 2.5 A 2×2 Cobb–Douglas Equilibrium Model: Cost Minimization and Marginal Utility

```
# GEMB high-level framework:
# firm uses CostMinimizationKKTConditions;
# consumer uses MarginalUtilityConsumerConditions.
using GeneralEquilibriumModeling

alpha = 2.0
omega = 100.0
beta = [0.5, 0.5]

production_function =
    x -> alpha * sqrt(x[1] * x[2])

marginal_product_function =
    x -> [
        0.5 * alpha * sqrt(x[2] / x[1]),
        0.5 * alpha * sqrt(x[1] / x[2]),
    ]

model = GEMB.GEMBModel(
    [:product, :labor];
    numeraire=:product,
)

GEMB.add_agent!(
    model,
    GEMB.ProductionFunctionSpec(
        production_function,
        marginal_product_function;
        behavior=:cost_min,
    );
    outputs=:product,
    demands=[:product, :labor],
    activity_start=100.0,
    demand_start=[100.0, 100.0],
    name=:firm,
)

GEMB.add_agent!(
    model,
    GEMB.CESMarginalUtilitySpec(beta; es=1.0);
    demands=[:product, :labor],
    endowments=:labor,
    endowment_quantities=omega,
    demand_start=[100.0, 100.0],
    demand_lower_bounds=[1.0e-10, 1.0e-10],
    multiplier_start=0.01,
    name=:consumer,
)

result = GEMB.solve(
```

2.7 Production Stationarity Conditions and Explicit Agent Conditions

`ProductionStationarityConditions` is used to represent a firm's production stationarity conditions for a given production technology. It incorporates the production function, marginal products, and their relationships with commodity prices directly into the equilibrium system, so that the firm's output level, input quantities, and associated production multiplier jointly satisfy the relevant first-order conditions.

Unlike `CostMinimizationKKTConditions`, `ProductionStationarityConditions` does not require the firm to minimize cost conditional on a given output level. Its economic interpretation is nevertheless closely related to the underlying production technology: for each input that is actually used, its marginal contribution must be consistent with the corresponding commodity price, while actual output must be consistent with the output determined by the production function. Thus, `ProductionStationarityConditions` provides a way to construct a firm's equilibrium conditions directly from the first-order stationarity relations of its production technology. The current formulation assumes an interior solution, so the marginal-pricing conditions are imposed as equalities.

`ExplicitAgentConditions` allows users to define an economic agent's equilibrium conditions directly. When existing condition rules such as `UnitRevenueExpenditureBalanceConditions`, `CostMinimizationKKTConditions`, or `MarginalUtilityConsumerConditions` cannot conveniently represent a particular economic relationship, users can supply their own condition functions and pair them with the agent's endogenous variables. `ExplicitAgentConditions` therefore provides substantial modeling flexibility and is useful for representing special behavioral constraints, institutional rules, and other nonstandard equilibrium conditions.

The following basic example illustrates the use of production stationarity conditions together with explicit agent conditions.

Listing 2.6 A 2×2 Cobb–Douglas Equilibrium Model: Production Stationarity Conditions and Explicit Agent Conditions

```
# production stationarity conditions and explicit consumer conditions.
using GeneralEquilibriumModeling

alpha = 2.0
omega = 100.0
beta = [0.5, 0.5]

production_function = x -> alpha * sqrt(x[1] * x[2])

marginal_product_function = x -> [
    0.5 * alpha * sqrt(x[2] / x[1]),
    0.5 * alpha * sqrt(x[1] / x[2]),
]

model = GEMB.GEMBModel(
    [:product, :labor];
    numeraire=:product,
    numeraire_value=1.0,
)

GEMB.add_agent!(
    model,
    GEMB.ProductionFunctionSpec(
        production_function,
        marginal_product_function;
        behavior=:stationary,
    );
    outputs=:product,
    demands=[:product, :labor],
    activity_start=100.0,
    demand_start=[100.0, 100.0],
    demand_lower_bounds=[1.0e-10, 1.0e-10],
    production_multiplier_start=1.0,
    name=:firm,
)

consumer_net_supply = function (v, p, observed_values=Any[])
    [-v[1], omega - v[2]]
end

consumer_conditions = function (v, p, net_supply)
    x = v[1:2]
    mu = v[3]

    return [
        mu * p[1] - beta[1] / x[1],
        mu * p[2] - beta[2] / x[2],
        sum(p .* net_supply),
    ]
end
```

2.8 Activity-Demand Specifications

An important class of behavioral specifications in the GEMB high-level framework is the *activity-demand specification*. It applies to economic agents whose commodity demands can be expressed directly as functions of an activity level and commodity prices.

For a class of homogeneous consumers represented without aggregation, the activity level may be interpreted as the number, or population, of identical consumers. This interpretation is common in strict-sense structural equilibrium models, where identical consumers need not be aggregated. In computational models, however, homogeneous consumers are usually aggregated into a representative consumer. When such a representative consumer is modeled through a utility-based demand specification, its activity level typically represents the utility level rather than the number of consumers.

Let z denote the activity level of an agent, $\mathbf{p}$ the relevant price vector, and $\mathbf{x}$ its commodity demand vector. An activity-demand specification can be written abstractly as

$$\mathbf{x} = \mathbf{d}(z, \mathbf{p})$$

where $\mathbf{d}$ is an activity-demand function. In more general cases, the demand function may also depend on observed variables.

The interpretation of the activity level depends on the type of economic agent. For a producer, the activity level usually represents its production level or production scale. For example, if a producer has constant returns to scale and requires the unit demand bundle $\mathbf{a}(\mathbf{p})$ at prices $\mathbf{p}$, its demand can be written as

$$\mathbf{x} = z\mathbf{a}(\mathbf{p})$$

where z is the production activity level.

The same representation can also be used for consumers. Suppose a consumer has a utility function, and let $\mathbf{x}^h(\mathbf{p}, u)$ denote its compensated demand at prices $\mathbf{p}$ and utility level u . Then

$$\mathbf{x} = \mathbf{x}^h(\mathbf{p}, u)$$

can be written in the same mathematical form as an activity-demand function. The consumer's utility level u can therefore be regarded as its **activity level**. This is the same general concept of an activity level, although its economic interpretation depends on how the consumer is represented. For an unaggregated class of homogeneous consumers, the activity level may instead represent the number or population of consumers.

This common representation makes it possible to treat producers and compensated-demand consumers within the same activity-demand framework. The essential distinction lies in the economic interpretation of the activity variable and in the conditions determining its equilibrium value, rather than in the mathematical form of the demand function.

In `GeneralEquilibriumModeling.jl`, activity-demand specifications belong to the abstract type `AbstractActivityDemandSpec`. The general-purpose specification is `ActivityDemandSpec`, whose demand function accepts either

```
demand_function(activity, prices)
```

or

```
demand_function(activity, prices, observed_values)
```

and returns the corresponding commodity demand vector. Thus, `ActivityDemandSpec` can be used whenever conditional commodity demand is known explicitly as a function of an activity level and prices.

GEMB also provides several built-in activity-demand specifications. `CESSpec` represents CES activity-demand relationships, while `DCESSpec` extends this representation to displaced CES relationships. These specifications can be used for both producers and compensated-demand consumers. `PowerProductionSpec` represents a one-input power production technology and is intended specifically for producers.

When an activity-demand specification is used for a producer, the activity variable must be associated with an appropriate producer condition. For homogeneous activity-demand relationships without fixed endowments, GEMB can use unit revenue-expenditure balance conditions. For specifications involving displacement terms or fixed endowments, total revenue-expenditure balance conditions are generally more appropriate. These defaults can be overridden when an alternative condition rule is required.

For a compensated-demand consumer, the construction is different. The consumer has no output commodity in the producer sense. Its net supply is obtained from its endowment minus its compensated demand, and its equilibrium utility activity is determined together with a total expenditure-income balance condition. Consequently, the activity-demand framework provides a unified way to describe two apparently different economic behaviors:

- a producer chooses a production activity level and demands the commodities required to support that production level;
- a consumer attains a utility activity level and demands the commodities required to support that utility level.

This interpretation is especially useful in structural equilibrium modeling because both production and consumption can be represented through the relationship between an activity level, commodity prices, and the associated demand bundle.

The function `activity_supply` is the common output-supply interface for activity-based producers in GEMB. It maps an activity level, output prices, and optionally observed equilibrium variables into the corresponding output quantities. Built-in output specifications such as `CETSpec` implement this interface automatically, while users can define new specifications by creating a subtype of `AbstractActivitySupplySpec` and extending `activity_supply`. Such a custom specification can then be supplied through the `output_spec` argument when constructing a producer.

2.9 Custom Net-Supply Agents and Condition Agents

2.9.1 Custom Net-Supply Agents and Multi-Activity Producers

The standard behavioral specifications in the GEMB high-level framework, such as `CESSpec`, `DCESSpec`, and `MarshallDemandConsumerSpec`, provide convenient representations for commonly used forms of economic behavior. When an agent cannot be represented naturally by one of these specifications, GEMB also allows its net-supply mapping to be defined directly through `add_net_supply_agent!`.

A custom net-supply agent may have its own endogenous variables, variable bounds and starting values, observed equilibrium variables, and condition rule. Its behavior is specified directly by a net-supply function that returns positive quantities for supplies and negative quantities for demands. This provides a flexible high-level interface for economic agents whose behavior does not fit one of the standard GEMB specifications.

A custom net-supply producer may also contain multiple endogenous activity levels. For example, a single firm may control an activity vector $\mathbf{z} = (z_1, \dots, z_m)^\top$ representing several production processes. When `UnitRevenueExpenditureBalanceConditions` is used, one unit revenue-expenditure balance condition is constructed for each activity variable. Thus, the number of production activities need not equal the number of economic agents. By contrast, the current standard activity-demand producer specifications, including `CESSpec` and `DCESSpec`, construct one activity level for each agent.

2.9.2 Condition Agents

A second extension mechanism is provided by `ConditionAgentSpec`. A condition agent provides the GEMB high-level representation of endogenous auxiliary variables together with the explicit conditions that determine them. It may contain one or more endogenous equilibrium variables, with one condition paired with each variable, but it does not generate commodity

demand or supply. It is therefore useful for representing variables that would typically be introduced through auxiliary variable–equation pairs in the GEM framework, including closure variables, endogenous policy parameters, normalization variables, and other equilibrium variables determined by explicit conditions.

2.9.3 A Two-Commodity von Neumann Model with Three Activities

The following von Neumann model illustrates how a custom net-supply agent and a condition agent can be used together.

Consider a pure-production economy with two commodities, wheat and iron, and three production processes (Li, 2019, Example 2.3). The input and output coefficient matrices are

$$\mathbf{A} = \begin{pmatrix} 0.8 & 0.5 & 0.06 \\ 2 & 2 & 0.4 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Each column represents a production process. Processes 1 and 2 both produce wheat, whereas process 3 produces iron. Processes 1 and 2 therefore compete in wheat production. Let $\mathbf{z} = (z_1, z_2, z_3)^\top$ denote the activity-level vector, $\mathbf{p} = (p_1, p_2)^\top$ the commodity-price vector, and ρ the equilibrium discount factor.

Processes 1 and 2 produce the same output, one unit of wheat, and use the same amount of iron, but process 1 requires 0.8 units of wheat as input whereas process 2 requires only 0.5. Process 1 therefore has a higher unit cost for the same unit revenue and is technologically disadvantaged.

Taking wheat as the numeraire, one equilibrium is

$$\mathbf{p}^* = \begin{pmatrix} 1 \\ 0.15 \end{pmatrix}, \quad \mathbf{z}^* = \begin{pmatrix} 0 \\ 100 \\ 500 \end{pmatrix}, \quad \rho^* = 0.8$$

The corresponding balanced-growth factor is $1/\rho^* = 1.25$, so the equilibrium growth rate is

$$\frac{1}{\rho^*} - 1 = 0.25$$

In the GEMB high-level implementation, the three production processes are represented as three activity variables of a single custom net-supply producer. Its aggregate net supply is defined as

$$(\rho\mathbf{B} - \mathbf{A})\mathbf{z}$$

The producer is constructed with `add_net_supply_agent!` and uses `UnitRevenueExpenditureBalanceCondition`. Consequently, activity j is paired with the unit excess cost

$$\mathbf{p}^\top \mathbf{A}_{.j} - \rho \mathbf{p}^\top \mathbf{B}_{.j}$$

A positive activity level therefore requires the corresponding unit excess cost to be zero, whereas a production process with a positive unit excess cost may remain inactive.

The equilibrium discount factor ρ is represented as the endogenous variable of a `ConditionAgentSpec`. Since the von Neumann activity vector is homogeneous, its scale is fixed by the normalization

$$z_1 + z_2 + z_3 = 600$$

which is used as the condition paired with ρ . Together with the price normalization $p_1 = 1$, the model simultaneously determines the relative commodity price, the three activity levels, and the equilibrium discount factor.

Thus, this example demonstrates two complementary extensions of the GEMB high-level framework: `add_net_supply_agent!` allows the direct construction of a multi-activity producer, while `ConditionAgentSpec` represents an endogenous auxiliary variable and the explicit condition that determines it.

Listing 2.7 A Two-Commodity Von Neumann Equilibrium Model with Three Activities

```
using GeneralEquilibriumModeling

A = [
    0.8  0.5  0.06
    2.0  2.0  0.40
]

B = [
    1.0  1.0  0.0
    0.0  0.0  1.0
]

model = GEMB.GEMBModel(
    [:wheat, :iron];
    numeraire=:wheat,
    numeraire_value=1.0,
)

GEMB.add_net_supply_agent!(
    model,
    function (variables, prices, observed_values)
        z = variables
        rho = observed_values[1]

        return (rho .* B - A) * z
    end;
    commodities=[:wheat, :iron],
    variable_names=[:activity1, :activity2, :activity3],
    variable_lower_bounds=[0.0, 0.0, 0.0],
    variable_upper_bounds=[Inf, Inf, Inf],
    variable_start=[100.0, 100.0, 100.0],
    observed_variables=[
        GEM.AgentVariableRef(:growth_condition, :rho),
    ],
    condition_rule=GEM.UnitRevenueExpenditureBalanceConditions(),
    name=:firm,
)

GEMB.add_agent!(
    model,
    GEMB.ConditionAgentSpec(
        (variables, observed_values) -> [
            sum(observed_values) - 600.0,
        ],
    ),
    variable_names=[:rho],
    variable_start=[0.8],
    variable_lower_bounds=[0.0],
    variable_upper_bounds=[Inf],
    observed_variables=[
        GEM.AgentVariableRef(:firm, :activity1),
    ],
    39
);
```

2.10 Equilibrium Results and Statistics

After a GEMB model has been constructed, it can normally be solved directly by

```
result = GEMB.solve(model)
```

The resulting object contains the equilibrium solution and solver information. For economic analysis, GEMB also provides post-solve functions that organize the solution into prices, activity levels, demand and supply matrices, and related value statistics.

2.10.1 Equilibrium Statistics

General equilibrium statistics can be obtained by

```
stats = GEMB.equilibrium_statistics(model, result)
```

The equilibrium price vector is stored in `stats.prices`. Explicitly represented activity levels are collected in `stats.activity_levels`, while `stats.activity_level_refs` identifies the corresponding agents. The economic interpretation of an activity level depends on the representation of the agent. For example, it may represent a production activity level or a consumer's utility level. A multi-activity agent may therefore contribute several entries to `activity_levels`, whereas an agent without an explicitly represented activity level contributes none.

GEMB represents each economic agent internally through its net supply. Let there be n commodities and m agents, and let the demand and supply matrices be

$$\mathbf{D} = [d_{ij}] \quad \text{and} \quad \mathbf{S} = [s_{ij}]$$

where rows correspond to commodities and columns correspond to agents. The net-supply matrix is defined as

$$\bar{\mathbf{S}} = \mathbf{S} - \mathbf{D}$$

Thus, $\bar{s}_{ij} > 0$ means that agent j is a net supplier of commodity i , while $\bar{s}_{ij} < 0$ means that the agent is a net demander of that commodity. The total net supply of each commodity is

$$\bar{\mathbf{S}}\mathbf{1} = \mathbf{S}\mathbf{1} - \mathbf{D}\mathbf{1}$$

The corresponding information is available from `stats.net_supply_matrix` and `stats.total_net_supply`.

The net-supply value matrix at the equilibrium price vector $\mathbf{p}$ is

$$\bar{\mathbf{S}}_v = \text{Diag}(\mathbf{p})\bar{\mathbf{S}}$$

and is available from `stats.net_supply_value_matrix`. Summing this matrix over commodities gives the net-supply value of each agent,

$$\bar{\mathbf{S}}^T \mathbf{p}$$

which is stored in `stats.agent_net_supply_values`. For an agent satisfying a revenue-expenditure balance condition, this value should be zero up to numerical error.

2.10.2 Demand and Supply Matrices

For models consisting of agents with activity-demand specifications, the demand and supply components can be obtained separately by

```
flows = GEMB.demand_supply_matrices(model, result)
```

The main matrices are

```
D = flows.demand
S = flows.supply
Sbar = flows.net_supply

DV = flows.demand_value
SV = flows.supply_value
```

where `D`, `S`, and `Sbar` correspond respectively to $\mathbf{D}$, $\mathbf{S}$, and $\bar{\mathbf{S}}$. Hence,

```
S - D    Sbar
```

up to numerical error.

The total demand and supply vectors are

$$\mathbf{d} = \mathbf{D}\mathbf{1} \quad \text{and} \quad \mathbf{s} = \mathbf{S}\mathbf{1}$$

and are available as `flows.total_demand` and `flows.total_supply`. Their relationship reflects the commodity-market conditions imposed in the model. In an equality-form equilibrium model,

$$\mathbf{D}\mathbf{1} = \mathbf{S}\mathbf{1}$$

whereas a model allowing free disposal or other inequality-form market conditions may permit positive total net supply for some commodities.

GEMB also reports demand and supply in value terms. Define

$$\mathbf{D}\mathbf{V} = \text{Diag}(\mathbf{p})\mathbf{D}$$

and

$$\mathbf{S}\mathbf{V} = \text{Diag}(\mathbf{p})\mathbf{S}$$

The element DV_{ij} is agent j 's expenditure on commodity i , while SV_{ij} is the revenue associated with its supply of that commodity. Consequently, the expenditure and revenue vectors of the agents are

$$\mathbf{e} = \mathbf{D}^\top \mathbf{p}$$

and

$$\mathbf{r} = \mathbf{S}^\top \mathbf{p}$$

These quantities are available as `flows.agent_expenditure` and `flows.agent_revenue`. Their difference satisfies

$$\mathbf{r} - \mathbf{e} = \bar{\mathbf{S}}^\top \mathbf{p}$$

Thus, the quantity matrices and value matrices provide two complementary views of an equilibrium: the former describe the physical allocation of commodities, while the latter describe the corresponding revenue and expenditure flows.

2.10.3 Printing and Checking an Equilibrium

For routine inspection, GEMB provides

```
GEMB.print_equilibrium_statistics(model, result)
```

which prints a compact summary of equilibrium prices, activity levels, net supplies, net-supply values, and aggregate net supplies. Numerical values close to zero can be suppressed in the display by specifying `display_tol`, and the number of significant digits can be controlled by `sigdigits`.

A typical post-solve workflow is therefore

```
result = GEMB.solve(model)

stats = GEMB.equilibrium_statistics(model, result)
flows = GEMB.demand_supply_matrices(model, result)

D = flows.demand
S = flows.supply
Sbar = flows.net_supply

DV = flows.demand_value
SV = flows.supply_value

@assert S - D == Sbar

GEMB.print_equilibrium_statistics(model, result)
```

For an equality-form model in which all agents satisfy revenue-expenditure balance, two additional numerical checks are

```
@assert flows.total_demand == flows.total_supply
@assert flows.agent_expenditure == flows.agent_revenue
```

These checks help distinguish the economic interpretation of an equilibrium from the numerical solution process. `solve` determines an equilibrium of the underlying mixed complementarity problem, while the statistics functions reorganize the resulting variables into economically meaningful quantities and provide convenient consistency checks.

3 Static Pure-Exchange Equilibrium

3.1 CES and DCES Consumers: A 2×2 Pure-Exchange Model

3.1.1 CES and DCES Specifications

CES and displaced CES functions are two built-in function families in the GEMB high-level framework. They are represented by `CESSpec` and `DCESpec`, respectively. Both are activity-demand specifications: given an activity level and the prices of demanded commodities, GEMB computes the corresponding conditional demand vector. For a producer, the activity level represents output or production scale; for a consumer, it can represent utility.

A CES specification is constructed by

```
GEMB.CESSpec(  
  beta;  
  es=1.0,  
  alpha=1.0,  
)
```

where `beta` is the vector of distribution parameters, `es` is the elasticity of substitution, and `alpha` is the scale parameter. Let $\beta = (\beta_1, \dots, \beta_n)^\top$, e_s denote `es`, and $\alpha > 0$. For $e_s \neq 0, 1$, the corresponding CES function takes the form

$$f(\mathbf{x}) = \alpha \left[\sum_{i=1}^n \beta_i x_i^{(e_s-1)/e_s} \right]^{e_s/(e_s-1)}$$

The parameter `es` directly represents the elasticity of substitution. When `es=1`, GEMB uses the Cobb–Douglas limit and requires $\sum_i \beta_i = 1$. When `es=0`, it uses the Leontief limit. Hence a single `CESSpec` interface covers the Leontief, Cobb–Douglas, and ordinary CES cases.

For example,

```
spec = GEMB.CESSpec(
    [0.4, 0.6];
    es=1.0,
    alpha=1.0,
)
```

defines a two-commodity Cobb–Douglas specification. Changing only **es** changes the degree of substitution without changing the general model-building interface.

If the activity level is z and the relevant price vector is $\mathbf{p}$, GEMB evaluates the conditional demand as

```
GEMB.activity_demand(spec, z, p)
```

For $e_s \neq 0, 1$, its i th component is

$$x_i = \frac{z}{\alpha} \beta_i^{e_s} p_i^{-e_s} \left(\sum_{j=1}^n \beta_j^{e_s} p_j^{1-e_s} \right)^{e_s/(1-e_s)}$$

When **CESSpec** is used for a consumer, the same conditional-demand relation is interpreted as compensated demand, with the activity level representing utility.

DCESSpec extends this construction by introducing a displacement vector (Fullerton, 1989). It is constructed by

```
GEMB.DCESSpec(
    beta;
    es=1.0,
    alpha=1.0,
    xi=zeros(length(beta)),
)
```

where **xi** is the displacement vector $\xi = (\xi_1, \dots, \xi_n)^\top$. For $e_s > 0$ and $e_s \neq 1$, the DCES function is parameterized as

$$f(\mathbf{x}) = \alpha \left[\sum_{i=1}^n \beta_i^{1/e_s} (x_i - \xi_i)^{(e_s-1)/e_s} \right]^{e_s/(e_s-1)}$$

When $e_s \rightarrow 0$, the DCES function converges to the displaced Leontief form

$$f(\mathbf{x}) = \alpha \min_{i: \beta_i > 0} \left\{ \frac{x_i - \xi_i}{\beta_i} \right\}$$

When $e_s = 1$, the corresponding Cobb–Douglas limit is used.

The displacement parameters shift the quantities entering the CES aggregator. The conditional demand consequently has the form

$$\mathbf{x} = \xi + \frac{z}{\alpha} \mathbf{a}(\mathbf{p})$$

where $\mathbf{a}(\mathbf{p})$ is the price-dependent unit demand vector. In particular, for $e_s \neq 1$,

$$x_i = \xi_i + \frac{z}{\alpha} \beta_i p_i^{-e_s} \left(\sum_{j=1}^n \beta_j p_j^{1-e_s} \right)^{e_s/(1-e_s)}$$

`CESSpec` and `DCESSpec` use different parameterizations of the CES coefficients. Consequently, setting `xi=zeros(...)` in a `DCESSpec` produces a homogeneous CES-family specification, but it should not in general be interpreted as numerically identical to a `CESSpec` constructed with the same values of `beta` and `alpha`. The two specifications should therefore be regarded as related built-in parameterizations rather than as interchangeable constructors.

When these specifications are passed to `GEMB.add_agent!`, `GEMB` combines the demand specification with the agent’s outputs, endowments, claims, and equilibrium conditions. For a `CESSpec` producer without fixed endowments, the default condition is `UnitRevenueExpenditureBalanceConditions`. A producer with fixed endowments instead uses `TotalRevenueExpenditureBalanceConditions`. For `DCESSpec`, a producer with a nonzero displacement parameter also uses `TotalRevenueExpenditureBalanceConditions`, since its net supply is no longer proportional to the activity level. These defaults can be replaced by explicitly supplying another condition rule.

Claims are not part of the CES or DCES function itself. If an ad valorem claim is required, it is attached separately when the agent is constructed through arguments such as `claim` and `claim_rate`. This separation allows the same CES or DCES behavioral specification to be reused under different institutional arrangements.

3.1.2 A 2×2 Pure-Exchange Model

The following pure-exchange model illustrates how `CESSpec` and `DCESSpec` can be used simultaneously for different consumers. Assume the following:

1. The economy contains two commodities and two consumers. Commodity 2 is taken as the numeraire commodity.

2. Consumer 1 is endowed with $\omega_1 > 0$ units of commodity 1 and no commodity 2. The consumer has a CES specification with **beta**=[**beta**, 1-**beta**], **es**=1, and **alpha**=1, where $0 < \beta < 1$. Thus, the CES specification reduces to the Cobb–Douglas case.
3. Consumer 2 is endowed with $\omega_2 > 0$ units of commodity 2 and no commodity 1. The consumer has a DCES specification with the same **beta**, **es**, and **alpha**, but with displacement vector $\xi = (0, \xi)^\top$, where $0 \leq \xi < \omega_2$.

Let the price vector be $\mathbf{p} = (p, 1)^\top$. The demand and supply matrices are

$$\mathbf{D}(p) = \begin{bmatrix} \beta\omega_1 & \frac{\beta(\omega_2 - \xi)}{p} \\ (1 - \beta)p\omega_1 & \xi + (1 - \beta)(\omega_2 - \xi) \end{bmatrix}$$

and

$$\mathbf{S} = \begin{bmatrix} \omega_1 & 0 \\ 0 & \omega_2 \end{bmatrix}$$

where rows represent commodities and columns represent consumers.

The equality-form wide-sense structural equilibrium model requires the revenue-expenditure balance condition

$$\mathbf{p}^\top \mathbf{D}(p) = \mathbf{p}^\top \mathbf{S}$$

and the supply-demand balance condition

$$\mathbf{D}(p)\mathbf{1} = \mathbf{S}\mathbf{1}$$

The equilibrium price vector is

$$\mathbf{p}^* = \left(\frac{\beta(\omega_2 - \xi)}{(1 - \beta)\omega_1}, 1 \right)^\top$$

The equilibrium demand matrix is

$$\mathbf{D}^* = \begin{bmatrix} \beta\omega_1 & (1 - \beta)\omega_1 \\ \beta(\omega_2 - \xi) & \xi + (1 - \beta)(\omega_2 - \xi) \end{bmatrix}$$

For example, let $\beta = 0.5$, $\omega_1 = \omega_2 = 100$, and $\xi = 20$. Then

$$\mathbf{p}^* = (0.8, 1)^\top$$

and

$$\mathbf{D}^* = \begin{bmatrix} 50 & 50 \\ 40 & 60 \end{bmatrix}, \quad \mathbf{S} = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix}$$

The first consumer therefore consumes $(50, 40)^\top$, while the second consumes $(50, 60)^\top$. The 20-unit displacement in commodity 2 directly illustrates how `DCESSpec` differs from an ordinary `CESSpec`.

The corresponding GEMB implementation is given in Listing [3.1](#).

Listing 3.1 CES and DCES Consumers: A 2×2 Pure-Exchange Model

```
using GeneralEquilibriumModeling

beta = [0.5, 0.5]

model = GEMB.GEMBModel(
    [:commodity1, :commodity2];
    numeraire=:commodity2,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec(
        beta;
        es=1.0,
        alpha=1.0,
    );
    demands=[:commodity1, :commodity2],
    endowments=:commodity1,
    endowment_quantities=100.0,
    activity_start=100.0,
    name=:consumer1,
)

GEMB.add_agent!(
    model,
    GEMB.DCESSpec(
        beta;
        es=1.0,
        alpha=1.0,
        xi=[0.0, 20.0],
    );
    demands=[:commodity1, :commodity2],
    endowments=:commodity2,
    endowment_quantities=100.0,
    activity_start=100.0,
    name=:consumer2,
)

result = GEMB.solve(
    model;
    p0=[1.0, 1.0],
    silent=true,
)

GEMB.print_equilibrium_statistics(model49, result)

flows = GEMB.demand_supply_matrices(model, result)

D = flows.demand
S = flows.supply
```

3.2 Marshallian-Demand Consumers: A 2×2 Pure-Exchange Model

When a consumer's Marshallian demand is available explicitly as a function of income and prices, GEMB allows the demand function to be supplied directly through `MarshallDemandConsumerSpec`. This avoids introducing a utility function, marginal-utility conditions, or a utility activity variable.

A Marshallian-demand specification is constructed as

```
GEMB.MarshallDemandConsumerSpec(demand_function)
```

where `demand_function` takes income and the relevant commodity prices as arguments and returns a demand vector. A typical function has the form

```
demand_function = (income, prices) -> begin
  # compute demands
  [...]
end
```

GEMB computes the consumer's income from the value of its endowments and then evaluates this demand function at the current prices. Thus, `MarshallDemandConsumerSpec` is particularly convenient when the demand system itself, rather than the underlying utility maximization problem, is the natural starting point of the model.

Unlike an activity-demand consumer, a consumer constructed from `MarshallDemandConsumerSpec` does not require an endogenous utility or activity variable. Its income is determined from its endowments at the current prices, and its commodity demands are then obtained directly from the supplied Marshallian-demand function. The consumer is therefore incorporated into the equilibrium system through its resulting net supply, while equilibrium prices are determined jointly by the commodity-market conditions.

This interface is useful not only for demands derived analytically from standard utility functions, but also whenever a Marshallian demand system has already been obtained independently and the researcher wishes to use that demand system directly in a general equilibrium model.

Consider the following pure-exchange economy:

1. The economy contains two commodities and two consumers. Commodity 2 is taken as the numeraire commodity.
2. Consumer 1 is endowed with $\omega_1 > 0$ units of commodity 1 and no commodity 2, while consumer 2 is endowed with $\omega_2 > 0$ units of commodity 2 and no commodity 1.

3. Consumer i allocates a share β_i of income to commodity 1 and the remaining share $1 - \beta_i$ to commodity 2, where $0 < \beta_i < 1$. Its Marshallian demand is therefore

$$\mathbf{x}_i(w_i, \mathbf{p}) = \left(\frac{\beta_i w_i}{p_1}, \frac{(1 - \beta_i) w_i}{p_2} \right)^\top$$

Let the price vector be $\mathbf{p} = (p, 1)^\top$. The incomes of the two consumers are $w_1 = p\omega_1$ and $w_2 = \omega_2$, respectively. Hence the demand and supply matrices are

$$\mathbf{D}(p) = \begin{bmatrix} \beta_1 \omega_1 & \frac{\beta_2 \omega_2}{p} \\ (1 - \beta_1) p \omega_1 & (1 - \beta_2) \omega_2 \end{bmatrix}$$

and

$$\mathbf{S} = \begin{bmatrix} \omega_1 & 0 \\ 0 & \omega_2 \end{bmatrix}$$

By solving the equality-form wide-sense structural equilibrium model, the equilibrium price vector is

$$\mathbf{p}^* = \left(\frac{\beta_2 \omega_2}{(1 - \beta_1) \omega_1}, 1 \right)^\top$$

For a numerical example, let $\beta_1 = 0.5$, $\beta_2 = 0.25$, and $\omega_1 = \omega_2 = 100$. The equilibrium is then

$$\mathbf{p}^* = (0.5, 1)^\top$$

and the equilibrium demand and supply matrices are

$$\mathbf{D}^* = \begin{bmatrix} 50 & 50 \\ 25 & 75 \end{bmatrix}, \quad \mathbf{S} = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix}$$

The corresponding GEMB implementation is given in Listing [3.2](#).

Listing 3.2 Marshallian-Demand Consumers: A 2×2 Pure-Exchange Model

```
using GeneralEquilibriumModeling

beta1 = 0.5
beta2 = 0.25
omega1 = 100.0
omega2 = 100.0

marshall_demand(beta) = (income, prices) -> [
    beta * income / prices[1],
    (1.0 - beta) * income / prices[2],
]

model = GEMB.GEMBModel(
    [
        GEMB.CommoditySpec(
            :commodity1;
            price_lower_bound=0.05,
        ),
        GEMB.CommoditySpec(
            :commodity2;
            price_lower_bound=0.05,
        ),
    ];
    numeraire=:commodity2,
)

GEMB.add_agent!(
    model,
    GEMB.MarshallDemandConsumerSpec(
        marshall_demand(beta1),
    );
    demands=[:commodity1, :commodity2],
    endowments=:commodity1,
    endowment_quantities=omega1,
    name=:consumer1,
)

GEMB.add_agent!(
    model,
    GEMB.MarshallDemandConsumerSpec(
        marshall_demand(beta2),
    );
    demands=[:commodity1, :commodity2],
    endowments=:commodity2,
    endowment_quantities=omega2,
    name=:consumer2,
)

result = GEMB.solve(
    model;
    p0=[1.0, 1.0],

```

3.3 Multiple Equilibria: A 2×2 Model

The example in this section (Mas-Colell, Whinston, Green, 1995, Exercise 15.B.6) illustrates the possibility of multiple equilibria: even with smooth, strictly monotone, and strictly convex CES preferences, a general equilibrium need not be unique.

Consider a pure exchange economy. Let $k = 12/37$, and assume the following:

1. The economy contains two commodities and two consumers.
2. Consumer 1 has the utility function $u_1(\mathbf{x}) = (x_1^{-2} + k^3 x_2^{-2})^{-1/2}$ and is endowed with one unit of commodity 1.
3. Consumer 2 has the utility function $u_2(\mathbf{x}) = (k^3 x_1^{-2} + x_2^{-2})^{-1/2}$ and is endowed with one unit of commodity 2.

Let the price vector be $\mathbf{p} = (p_1, p_2)^\top$, and define

$$t = \left(\frac{p_1}{p_2} \right)^{1/3}$$

The incomes of consumers 1 and 2 are $w_1 = p_1$ and $w_2 = p_2$, respectively. Utility maximization yields the following demand and supply matrices:

$$\mathbf{D}(t) = \begin{bmatrix} \frac{t^2}{t^2+k} & \frac{k}{t(kt^2+1)} \\ \frac{kt^3}{t^2+k} & \frac{1}{kt^2+1} \end{bmatrix}, \quad \mathbf{S} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Using the following equality-form wide-sense structural equilibrium model,

$$\mathbf{p}^\top \mathbf{D}(t) = \mathbf{p}^\top \mathbf{S}$$

$$\mathbf{D}(t)\mathbf{1} = \mathbf{S}\mathbf{1}$$

we obtain

$$(t-1)[kt^2 + (k-1)t + k] = 0$$

Substituting $k = 12/37$ yields

$$(t-1)(4t-3)(3t-4) = 0$$

and hence

$$t = \frac{3}{4}, \quad 1, \quad \frac{4}{3}$$

Since

$$\frac{p_1}{p_2} = t^3$$

the three equilibrium relative prices are

$$\frac{27}{64}, \quad 1, \quad \frac{64}{27}$$

Taking commodity 2 as the numeraire, so that $p_2 = 1$, the three equilibrium price vectors are

$$\mathbf{p}^{(1)} = \left(\frac{27}{64}, 1 \right)^\top, \quad \mathbf{p}^{(2)} = (1, 1)^\top, \quad \mathbf{p}^{(3)} = \left(\frac{64}{27}, 1 \right)^\top$$

The corresponding equilibrium demand matrices are

$$\mathbf{D}^{(1)} = \begin{bmatrix} \frac{111}{175} & \frac{64}{175} \\ \frac{27}{175} & \frac{148}{175} \end{bmatrix}, \quad \mathbf{D}^{(2)} = \begin{bmatrix} \frac{37}{49} & \frac{12}{49} \\ \frac{12}{49} & \frac{37}{49} \end{bmatrix}, \quad \mathbf{D}^{(3)} = \begin{bmatrix} \frac{148}{175} & \frac{27}{175} \\ \frac{64}{175} & \frac{111}{175} \end{bmatrix}$$

For $k = 12/37$, this multiplicity takes a particularly simple form: the market-equilibrium equation has exactly three positive solutions for the relative price.

Listing 3.3 A 2×2 Equilibrium Model: Multiple Equilibria

```
using GeneralEquilibriumModeling.GEM
using GeneralEquilibriumModeling.GEMB

k = 12 / 37

model = GEMBModel(
    [:commodity1, :commodity2];
    numeraire=:commodity2,
)

add_agent!(
    model,
    CESSpec([1.0, k^3]; es=1 / 3);
    demands=[:commodity1, :commodity2],
    endowments=:commodity1,
    endowment_quantities=1.0,
    name=:consumer1,
)

add_agent!(
    model,
    CESSpec([k^3, 1.0]; es=1 / 3);
    demands=[:commodity1, :commodity2],
    endowments=:commodity2,
    endowment_quantities=1.0,
    name=:consumer2,
)

for p0 in (
    [0.30, 1.0],
    [0.80, 1.0],
    [1.20, 1.0],
    [3.00, 1.0],
)
    result = solve(model; p0=p0, residual_tol=1e-10, silent=true)
    println("p0 = ", p0, " => p* = ", result.prices)
end
```

3.4 Money Illusion and Absolute-Price-Dependent Utility: A 2×2 Model

In standard consumer theory, preferences are defined independently of prices. Accordingly, demand functions are homogeneous of degree zero in prices and income: if all prices and income are multiplied by the same positive factor, optimal demand remains unchanged. In this sense, only relative prices matter, and the absolute price level has no effect on the equilibrium allocation. A violation of this property may be interpreted as **money illusion**.

Consider the following absolute-price-dependent utility function:

$$u(\mathbf{x}; p_1) = (x_1 - 2p_1)x_2$$

Price-dependent preferences have also been studied in the literature (see Pollak, 1977). Restrict attention to positive prices and positive income. If all income is spent, the corresponding demand function is

$$\mathbf{x}(\mathbf{p}, w) = \begin{cases} \left(\frac{w}{2p_1} + p_1, \frac{w}{2p_2} - \frac{p_1^2}{p_2} \right)^\top, & w > 2p_1^2 \\ \left(\frac{w}{p_1}, 0 \right)^\top, & w \leq 2p_1^2 \end{cases}$$

For example, when $\lambda > 1$ and $w > 2\lambda p_1^2$, the first commodity demand satisfies $x_1(\lambda \mathbf{p}, \lambda w) = w/(2p_1) + \lambda p_1 \neq x_1(\mathbf{p}, w)$. Hence the demand function is not homogeneous of degree zero in prices and income. The absolute price level therefore affects consumer demand.

Now consider a pure-exchange economy with two commodities and two consumers. The supply matrix is

$$\mathbf{S} = \begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix}, \quad s_1, s_2 > 0$$

Consumer 1 owns commodity 1 and has utility function $u_1 = x_{21}$, so the consumer demands only commodity 2. Consumer 2 owns commodity 2 and has the absolute-price-dependent utility function introduced above. Let $\mathbf{p} = (p_1, p_2)^\top$, let u_1 denote consumer 1's utility level, and let w_2 denote consumer 2's income. Restrict attention to the interior case $w_2 > 2p_1^2$. The demand matrix is then

$$\mathbf{D}(\mathbf{p}, u_1, w_2) = \begin{pmatrix} 0 & \frac{w_2}{2p_1} + p_1 \\ u_1 & \frac{w_2}{2p_2} - \frac{p_1^2}{p_2} \end{pmatrix}$$

The equality-form wide-sense structural equilibrium model requires each consumer's revenue and expenditure to balance and each commodity market to clear. Because consumer 2's demand depends on the absolute value of p_1 , the equilibrium price vector cannot in general be rescaled without changing the allocation. Thus, unlike a standard Walrasian model, fixing one price here is not merely an economically neutral normalization.

Let $p_1^* = q$ and regard q as a given absolute price. The remaining endogenous variables are p_2 , u_1 , and w_2 . Consumer 1's revenue-expenditure balance condition is $p_2 u_1 = p_1 s_1$, while consumer 2's condition is $w_2 = p_2 s_2$. Commodity 2 market clearing requires

$$u_1 + \frac{w_2}{2p_2} - \frac{p_1^2}{p_2} = s_2$$

Solving these three conditions for a given q yields

$$\mathbf{p}^*(q) = \left(q, \frac{2q(s_1 - q)}{s_2} \right)^\top, \quad u_1^*(q) = \frac{s_1 s_2}{2(s_1 - q)}, \quad w_2^*(q) = 2q(s_1 - q)$$

The interior-solution condition $w_2^* > 2(p_1^*)^2$ implies $0 < q < s_1/2$. Over this interval, $0 < p_2^* < s_1^2/(2s_2)$. Hence the model possesses a continuum of equilibria indexed by the absolute price q . These equilibria are economically distinct rather than proportional representations of the same relative-price equilibrium.

The model can be calculated directly with the GEMB high-level framework using `MarshallDemandConsumerSpec`. For a selected value of q , the program fixes the absolute price $p_1 = q$ by setting `numeraire=:commodity1` and `numeraire_value=q`. Because consumer 2's demand depends on the absolute value of p_1 , this operation should be interpreted as fixing an absolute price rather than as a simple price normalization.

For consumer 1, GEMB automatically calculates income from the value of the endowment, $w_1 = p_1 s_1$, and the Marshall demand for commodity 2 is $x_{21} = w_1/p_2$. Hence consumer 1's utility is obtained directly as $u_1 = x_{21}$. For consumer 2, GEMB similarly calculates income as $w_2 = p_2 s_2$ and passes this income together with the current prices to the user-defined Marshall demand function

$$\mathbf{x}_2(\mathbf{p}, w_2) = \left(\frac{w_2}{2p_1} + p_1, \frac{w_2}{2p_2} - \frac{p_1^2}{p_2} \right)^\top$$

Thus, neither u_1 nor w_2 needs to be introduced as an additional endogenous agent variable. Given $p_1 = q$, GEMB constructs the consumers' net supplies from their endowments and Marshall demands and determines the remaining price p_2 from market clearing.

The program below takes $s_1 = s_2 = 4$ and solves the model for two different absolute prices. When $p_1 = q = 1$, the analytical solution is $\mathbf{p}^* = (1, 1.5)^\top$, $u_1^* = 8/3$, and $w_2^* = 6$. When

$p_1 = q = 1.5$, the analytical solution becomes $\mathbf{p}^* = (1.5, 1.875)^\top$, $u_1^* = 3.2$, and $w_2^* = 7.5$. Both values satisfy the interior-solution condition $0 < q < s_1/2$. The resulting changes in p_2 , u_1 , and w_2 demonstrate that changing the absolute value of p_1 selects economically different equilibria rather than merely rescaling the same equilibrium price vector.

Listing 3.4 A 2×2 Pure-Exchange Equilibrium with Absolute-Price-Dependent Utility

```
using GeneralEquilibriumModeling

const S1 = 4.0
const S2 = 4.0
const Q = 1.5

model = GEMB.GEMBModel(
    [:commodity1, :commodity2];
    numeraire=:commodity1,
    numeraire_value=Q,
)

consumer1_demand = (income, prices) -> [
    income / prices[1],
]

GEMB.add_agent!(
    model,
    GEMB.MarshallDemandConsumerSpec(consumer1_demand);
    demands=:commodity2,
    endowments=:commodity1,
    endowment_quantities=[S1],
    name=:consumer1,
)

consumer2_demand = function (income, prices)
    p1, p2 = prices

    return [
        income / (2p1) + p1,
        income / (2p2) - p1^2 / p2,
    ]
end

GEMB.add_agent!(
    model,
    GEMB.MarshallDemandConsumerSpec(consumer2_demand);
    demands=[:commodity1, :commodity2],
    endowments=:commodity2,
    endowment_quantities=[S2],
    name=:consumer2,
)

result = GEMB.solve(
    model;
    p0=[Q, 1.0],
    residual_tol=1.0e-10,
    silent=true,
)

println("Equilibrium prices: ", result.prices)
```

3.5 Relative-Price-Dependent Utility: A 2×2 Model

Consider the utility function

$$u(\mathbf{x}; \mathbf{p}) = x_1^{\frac{p_1^2}{p_1^2 + p_2^2}} x_2^{\frac{p_2^2}{p_1^2 + p_2^2}}$$

This function has the form of a Cobb–Douglas utility function, but its expenditure shares depend on the relative price p_1/p_2 . It is therefore a **relative-price-dependent utility function**. As the relative price of commodity 1 rises, the consumer assigns a larger weight to commodity 1.

Relative-price-dependent preferences can be used to represent several types of price-dependent behavior (see Pollak, 1977). For example, consumers may use prices as signals of commodity quality, a higher relative price may increase the attractiveness of a commodity through conspicuous consumption, or price movements may affect speculative demand.

Unlike the absolute-price-dependent utility function considered in the previous section, this utility function is unchanged when all prices are multiplied by the same positive factor. The preference therefore depends on relative rather than absolute prices. The associated Marshall demand is also homogeneous of degree zero in prices and income.

Consider a pure-exchange economy with two commodities and two consumers. Commodity 2 is the numeraire. The supply matrix is

$$\mathbf{S} = \begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix}, \quad s_1, s_2 > 0$$

Consumer 1 owns s_1 units of commodity 1 and has utility function $u_1 = x_{21}$, so this consumer demands only commodity 2. Consumer 2 owns s_2 units of commodity 2 and has the relative-price-dependent utility function introduced above.

Let $\mathbf{p} = (p_1, 1)^\top$. Consumer 1's income is $p_1 s_1$, so its demand for commodity 2, and hence its utility level, is $u_1 = p_1 s_1$. Consumer 2's income is $w_2 = s_2$, and its Marshall demands are $x_{12} = p_1 w_2 / (p_1^2 + 1)$ and $x_{22} = w_2 / (p_1^2 + 1)$. The demand matrix can therefore be written as

$$\mathbf{D}(\mathbf{p}, u_1, w_2) = \begin{pmatrix} 0 & \frac{p_1 w_2}{p_1^2 + 1} \\ u_1 & \frac{w_2}{p_1^2 + 1} \end{pmatrix}$$

For the equality-form wide-sense structural equilibrium model, market clearing for commodity 1 requires $p_1 s_2 / (p_1^2 + 1) = s_1$. Equivalently,

$$s_1 p_1^2 - s_2 p_1 + s_1 = 0$$

Hence the positive equilibrium prices are

$$\mathbf{p}^* = \left(\frac{s_2 \pm \sqrt{s_2^2 - 4s_1^2}}{2s_1}, 1 \right)^\top$$

with $w_2^* = s_2$ and $u_1^* = p_1^* s_1$.

The number of positive equilibria depends on the relative magnitudes of the two supplies. If $s_2 > 2s_1$, there are two positive equilibrium values of p_1 , and their product is one, so they are reciprocals. If $s_2 = 2s_1$, there is a unique positive equilibrium with $p_1^* = 1$. If $s_2 < 2s_1$, no positive price can clear both markets.

This result can also be seen directly from consumer 2's demand for commodity 1, $x_{12} = p_1 s_2 / (p_1^2 + 1)$. As a function of p_1 , this demand first rises and then falls, reaching its maximum value $s_2/2$ at $p_1 = 1$. Hence commodity 1 can be fully absorbed only if $s_1 \leq s_2/2$, or equivalently $s_2 \geq 2s_1$. When the inequality is strict, the horizontal supply level s_1 intersects this nonmonotonic demand function twice, producing two positive equilibrium prices.

The model can be calculated directly with the GEMB high-level framework using `MarshallDemandConsumerSpec`. Since commodity 2 is the numeraire, the program sets `numeraire=:commodity2` and `numeraire_value=1.0`. GEMB automatically computes consumer 1's income from its endowment and price and then evaluates its Marshall demand for commodity 2. For consumer 2, GEMB similarly calculates $w_2 = p_2 s_2 = s_2$ and passes this income together with the current prices to the user-defined Marshall demand function. No additional income or utility variables need to be introduced.

An especially useful feature of this example is that the same GEMB model may have more than one positive equilibrium. For example, let $s_1 = 1$ and $s_2 = 3$. The two analytical equilibrium prices of commodity 1 are

$$p_{1L}^* = \frac{3 - \sqrt{5}}{2} \approx 0.381966, \quad p_{1H}^* = \frac{3 + \sqrt{5}}{2} \approx 2.618034$$

The corresponding utility levels of consumer 1 are $u_{1L}^* \approx 0.381966$ and $u_{1H}^* \approx 2.618034$, while consumer 2's income is $w_2^* = 3$ in both equilibria. The corresponding demand matrices are approximately

$$\mathbf{D}_L^* = \begin{pmatrix} 0 & 1 \\ 0.381966 & 2.618034 \end{pmatrix}, \quad \mathbf{D}_H^* = \begin{pmatrix} 0 & 1 \\ 2.618034 & 0.381966 \end{pmatrix}$$

Thus, the two equilibrium prices are not merely alternative numerical representations of the same allocation. They support two different equilibrium allocations. In numerical computation, the two equilibria can be obtained by solving the same GEMB model with initial price

vectors chosen near the lower-price and higher-price branches, respectively. This also provides a simple example of how the initial value supplied to a nonlinear equilibrium solver can determine which member of a multiple-equilibrium set is obtained.

If the equality-form wide-sense structural equilibrium model is replaced by the corresponding mixed-form wide-sense structural equilibrium model, an additional boundary equilibrium is possible:

$$\mathbf{p}^* = (0, 1)^\top, \quad u_1^* = 0, \quad w_2^* = s_2$$

At this equilibrium, both commodities satisfy the supply-demand balance conditions. For commodity 1, supply exceeds demand, but its equilibrium price is zero, so the supply-demand balance condition is satisfied through complementarity between the zero price and excess supply. For commodity 2, supply equals demand. Thus, the two commodity markets are both in supply-demand balance, although supply and demand are not equal for commodity 1. This boundary solution belongs to the mixed-form wide-sense structural equilibrium model rather than to the positive-price equality-form equilibrium considered above.

Listing 3.5 A 2×2 Pure-Exchange Model with Relative-Price-Dependent Utility and Multiple Equilibria

```
using GeneralEquilibriumModeling

const S1 = 1.0
const S2 = 3.0

model = GEMB.GEMBModel(
    [:commodity1, :commodity2];
    numeraire=:commodity2,
    numeraire_value=1.0,
)

consumer1_demand = (income, prices) -> [
    income / prices[1],
]

GEMB.add_agent!(
    model,
    GEMB.MarshallDemandConsumerSpec(consumer1_demand);
    demands=:commodity2,
    endowments=:commodity1,
    endowment_quantities=[S1],
    name=:consumer1,
)

consumer2_demand = function (income, prices)
    p1, p2 = prices

    return [
        p1 * income / (p1^2 + p2^2),
        p2 * income / (p1^2 + p2^2),
    ]
end

GEMB.add_agent!(
    model,
    GEMB.MarshallDemandConsumerSpec(consumer2_demand);
    demands=[:commodity1, :commodity2],
    endowments=:commodity2,
    endowment_quantities=[S2],
    name=:consumer2,
)

result_low = GEMB.solve(
    model;
    p0=[0.3, 1.0],
    residual_tol=1.0e-10,
    silent=true,
)

result_high = GEMB.solve(
```

3.6 Asset-Exchange Equilibrium with AMSD Utility: A 2×2 Model

Suppose there are n assets and m investors. Let $\mathbf{x}_i$ denote the portfolio held by investor i , μ_i the vector of subjective expected asset payoffs, and Σ_i the corresponding covariance matrix. Under additive mean-standard-deviation (AMSD) preferences, investor i has utility

$$U_i(\mathbf{x}_i) = \mu_i^\top \mathbf{x}_i - \gamma_i \sqrt{\mathbf{x}_i^\top \Sigma_i \mathbf{x}_i}$$

where $\gamma_i \geq 0$ is the risk-aversion coefficient (Nakamura, 2015). Thus, expected payoff increases utility, whereas portfolio risk measured by the standard deviation reduces utility.

`GeneralEquilibriumModeling.jl` provides the high-level function `GEMB.solve_asset_equilibrium_amsd` for computing this equilibrium. Short selling is excluded in the implementation. Suppose `Supply`, `PMP`, and `PSD` are $n \times m$ matrices containing initial asset endowments, subjective expected payoffs, and subjective payoff standard deviations, respectively; `Cor` is the $n \times n$ payoff correlation matrix; and `gamma` contains the m risk-aversion coefficients. The equilibrium can then be computed directly by

```
using GeneralEquilibriumModeling

result = GEMB.solve_asset_equilibrium_amsd(
    Supply=Supply,
    PMP=PMP,
    PSD=PSD,
    Cor=Cor,
    gamma=gamma,
)
```

For each investor, GEMB internally constructs a `MeanStandardDeviationMarginalUtilitySpec`, builds the corresponding pure-exchange consumer, and solves the resulting mixed complementarity problem through GEM. For numerical stability, the software uses a small smoothing parameter in the portfolio standard deviation.

By default, the last asset is used as the numeraire. The main results include `result.p` for equilibrium asset prices, `result.D` for the equilibrium holding matrix, and `result.lambda` for the investors' budget multipliers. Quantities such as `result.VMU`, `result.market_residual`, and `result.kkt_passed` can be used to examine the marginal-utility and equilibrium conditions.

Listing 3.6 A 2×2 Asset-Exchange Equilibrium Model

```
using GeneralEquilibriumModeling

Supply = [
    100.0  100.0
    100.0  100.0
]

PMP = [
    1.20  1.05
    1.05  1.20
]

PSD = [
    0.20  0.20
    0.30  0.30
]

Cor = [
    1.0  0.0
    0.0  1.0
]

gamma = [0.7, 1.0]

result = GEMB.solve_asset_equilibrium_amsd(
    Supply=Supply,
    gamma=gamma,
    PMP=PMP,
    PSD=PSD,
    Cor=Cor,
    x0=fill(100.0, 2, 2),
    silent=true,
)

println("Prices: ", result.p)
println("Holdings:")
display(result.D)
```

3.7 Matthew Effect in Asset-Exchange Equilibrium: A 2×2 Model

The Matthew effect refers to a process in which an initially small economic disparity becomes larger through subsequent interactions. In asset markets, such an effect may arise when investors' attitudes toward risk depend on their wealth. A relatively poor investor may become more risk averse and consequently hold more low-risk, low-expected-payoff assets, whereas a relatively rich investor may tolerate more risk and hold more high-risk, high-expected-payoff assets (Björk, Murgoci, Zhou, 2014). Asset exchange may therefore widen an initially small wealth difference.

The following example illustrates this mechanism using the AMSD preferences introduced in the previous section. There are two assets and two investors. Their initial asset-endowment matrix is

$$\mathbf{S} = \begin{pmatrix} 0.49 & 0.51 \\ 0.49 & 0.51 \end{pmatrix}$$

where rows represent assets and columns represent investors. Thus, investor 1 initially owns 0.49 units of each asset, whereas investor 2 owns 0.51 units of each asset.

Both investors have the same subjective expected asset payoffs,

$$\mu_1 = \mu_2 = \begin{pmatrix} 120 \\ 100 \end{pmatrix}$$

and the same payoff standard deviations,

$$\sigma_1 = \sigma_2 = \begin{pmatrix} 50 \\ 10 \end{pmatrix}$$

The two asset payoffs are assumed to be uncorrelated, and the investors' common correlation matrix is

$$\mathbf{R} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Asset 1 therefore has both the higher expected payoff and the higher risk, whereas asset 2 has the lower expected payoff and the lower risk.

The distinguishing feature of this example is that the risk-aversion coefficient depends on the investor's wealth. In the present example, expected portfolio payoff $w_i = \mu_i^\top \mathbf{x}_i$ is used as a simple indicator of investor i 's wealth. Taking 110 as the benchmark, set $\gamma_i = 0.8$ when $w_i < 110/1.02$, $\gamma_i = 0.4$ when $110/1.02 \leq w_i \leq 110 \times 1.02$, and $\gamma_i = 0.2$ when $w_i > 110 \times 1.02$.

Thus, lower expected wealth is associated with greater risk aversion, whereas higher expected wealth is associated with lower risk aversion.

Under the initial endowments, the expected portfolio payoffs are (107.8, 112.2), so the initial payoff gap is only 4.4. The corresponding initial risk-aversion coefficients are (0.8, 0.4).

For a given vector of risk-aversion coefficients, `GEMB.solve_asset_equilibrium_amsd` directly computes the asset-exchange equilibrium. The expected portfolio payoffs implied by the resulting holdings are then used to update the risk-aversion coefficients. This process is repeated until the risk-aversion coefficients are consistent with the equilibrium portfolios that they generate.

In this example, the procedure reaches a fixed point with $\gamma^* = (0.8, 0.2)$. Using asset 2 as the numeraire, the equilibrium asset-price vector is approximately

$$\mathbf{p}^* = \begin{pmatrix} 1.10036 \\ 1 \end{pmatrix}$$

and the equilibrium portfolio matrix is approximately

$$\mathbf{D}^* = \begin{pmatrix} 0.09245 & 0.90755 \\ 0.92745 & 0.07255 \end{pmatrix}$$

Investor 1, who starts with the lower expected wealth and has the higher risk-aversion coefficient, ends up holding mainly the lower-risk asset 2. Investor 2, who starts with the higher expected wealth and eventually has the lower risk-aversion coefficient, ends up holding mainly the higher-risk and higher-expected-payoff asset 1.

The equilibrium expected portfolio payoffs are approximately (103.839, 116.161). Hence the payoff gap increases from 4.4 to approximately 12.322. Asset exchange therefore amplifies the initial difference in expected wealth in this example. The mechanism is self-reinforcing: lower expected wealth is associated with greater risk aversion and a portfolio tilted toward the lower-risk and lower-expected-payoff asset, whereas higher expected wealth is associated with lower risk aversion and greater exposure to the higher-risk and higher-expected-payoff asset.

Computationally, the model has a simple two-step structure. Conditional on a given γ , `GEMB.solve_asset_equilibrium_amsd` solves the asset-exchange equilibrium directly. The wealth-dependent risk-aversion rule then updates γ , and the calculation continues until this rule and the asset-market equilibrium are mutually consistent.

Listing 3.7 A 2×2 Asset-Exchange Equilibrium Model Illustrating the Matthew Effect

```
using GeneralEquilibriumModeling

function gamma_from_payoff(expected_payoff, low_threshold, high_threshold)
    tol = 1.0e-10

    if expected_payoff > high_threshold + tol
        return 0.2
    elseif expected_payoff < low_threshold - tol
        return 0.8
    else
        return 0.4
    end
end

portfolio_expected_payoff(PMP, D) =
    vec(sum(PMP .* D; dims=1))

function main()
    Supply = [
        0.49 0.51
        0.49 0.51
    ]

    PMP = [
        120.0 120.0
        100.0 100.0
    ]

    PSD = [
        50.0 50.0
        10.0 10.0
    ]

    Cor = [
        1.0 0.0
        0.0 1.0
    ]

    payoff_benchmark = 110.0
    low_threshold = payoff_benchmark / 1.02
    high_threshold = payoff_benchmark * 1.02
    68

    initial_expected_payoff =
        portfolio_expected_payoff(PMP, Supply)

    gamma =
        gamma_from_payoff.(
            initial_expected_payoff,
```

4 Static Equilibrium with Production: Basic Models

4.1 Production and Transformation Specifications

GEMB provides several specifications for describing production technologies and the transformation of an activity level into outputs. These specifications can be divided into two broad classes. A production specification describes the relationship between production activity and inputs, whereas a transformation specification describes the relationship between activity level and outputs.

4.1.1 Production-Function Specification

`ProductionFunctionSpec` can be used when the production function and its marginal products are known explicitly. Let the production function be

$$y = f(\mathbf{x})$$

where $\mathbf{x} = (x_1, \dots, x_n)^\top$ is the input vector. The user supplies both $f(\mathbf{x})$ and a marginal-product vector. The latter may be either the exact gradient

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)^\top$$

or this vector multiplied by a common strictly positive factor, since such a scaling does not change the production stationarity conditions.

For example, consider the Cobb–Douglas production function

$$f(\mathbf{x}) = x_1^{0.6} x_2^{0.4}$$

Its marginal-product vector can be multiplied by the common positive factor $x_1^{0.4} x_2^{0.6}$, giving the simpler proportional vector $(0.6x_2, 0.4x_1)^\top$. The specification can therefore be written as

```

production_function = x ->
  x[1]^0.6 * x[2]^0.4

marginal_product_function = x -> [
  0.6 * x[2],
  0.4 * x[1],
]

spec = GEMB.ProductionFunctionSpec(
  production_function,
  marginal_product_function;
  behavior=:stationary,
)

```

The keyword `behavior` determines how the production technology is incorporated into the equilibrium system. With `behavior=:stationary`, GEMB constructs production stationarity conditions. The current stationary formulation assumes an interior solution, so the marginal-pricing conditions are imposed as equalities. With `behavior=:cost_min`, GEMB instead constructs cost-minimization KKT conditions, which allow corner solutions. Thus, the production function describes the technology, while `behavior` specifies the conditions imposed on the producer.

Commodity mappings are not contained in `ProductionFunctionSpec`. They are supplied when the producer is added to the model. For example,

```

GEMB.add_agent!(
  model,
  spec;
  outputs=:product,
  demands=[:labor, :capital],
  activity_start=100.0,
  demand_start=[100.0, 100.0],
  name=:firm,
)

```

The resulting producer contains an activity level, endogenous input quantities, and a production multiplier.

4.1.2 Power-Production Specification

For the commonly used one-input power production function

$$y = \alpha x^\theta, \quad \alpha > 0, \quad \theta > 0$$

GEMB provides the simpler built-in `PowerProductionSpec`:

```
GEMB.PowerProductionSpec(  
    alpha=1.0,  
    theta=1.0,  
)
```

If the production activity is y , the required input is obtained directly from the inverse production function,

$$x = \left(\frac{y}{\alpha}\right)^{1/\theta}$$

Thus, unlike `ProductionFunctionSpec`, `PowerProductionSpec` does not introduce the input quantity through a separate production-function constraint and marginal-product conditions. Instead, the input demand is computed directly as a function of the activity level. It is therefore an activity-demand specification and requires exactly one input commodity.

A producer can, for example, be constructed by

```
GEMB.add_agent!(  
    model,  
    GEMB.PowerProductionSpec(  
        alpha=2.0,  
        theta=0.5,  
    );  
    outputs=:product,  
    demands=:labor,  
    activity_start=100.0,  
    name=:firm,  
)
```

By default, a producer based on `PowerProductionSpec` uses `TotalRevenueExpenditureBalanceConditions`. Hence the default specification imposes total revenue-expenditure balance at a positive activity level. When $\theta \neq 1$, this condition should not be confused with marginal-cost pricing or the first-order condition of unconstrained profit maximization. Alternative producer conditions may be supplied explicitly when required by the economic model.

`ProductionFunctionSpec` and `PowerProductionSpec` therefore provide two different ways to represent production. The former explicitly introduces input quantities and production first-order conditions, whereas the latter substitutes the inverse production function directly into the producer's conditional demand.

4.1.3 CET Transformation Specification

When one production activity generates several possible outputs, their proportions need not be fixed. GEMB provides `CETSpec` for a constant-elasticity-of-transformation (CET) output specification:

```
GEMB.CETSpec(  
  beta;  
  et=1.0,  
  alpha=1.0,  
)
```

Let $\beta = (\beta_1, \dots, \beta_m)^\top$ denote the output weights, $\eta \geq 0$ denote the elasticity of transformation corresponding to `et`, and $\alpha > 0$ denote the scale parameter. For an output-price vector $\mathbf{p} = (p_1, \dots, p_m)^\top$, GEMB uses the unit-revenue function

$$r(\mathbf{p}) = \frac{1}{\alpha} \left(\sum_{i=1}^m \beta_i p_i^{1+\eta} \right)^{1/(1+\eta)}$$

If the activity level is z , the conditional supply of output i is

$$y_i = \frac{z}{\alpha} \beta_i p_i^\eta \left(\sum_{j=1}^m \beta_j p_j^{1+\eta} \right)^{-\eta/(1+\eta)}$$

Hence, for two outputs with positive quantities and prices,

$$\frac{y_i}{y_j} = \frac{\beta_i}{\beta_j} \left(\frac{p_i}{p_j} \right)^\eta$$

so `et` directly measures the elasticity of the output ratio with respect to the relative output price. A larger `et` therefore makes the output composition more responsive to relative prices. When `et=0`, the output proportions are fixed:

$$y_i = \frac{\beta_i}{\alpha} z$$

`CETSpec` is an output-side specification and is supplied through the `output_spec` argument of an activity-demand producer. For example, a producer may use a CES specification for its conditional input demands and a CET specification for its conditional output supplies:


```

GEMB.add_agent!(
  model,
  GEMB.CESSpec(
    [0.6, 0.4];
    es=1.0,
    alpha=1.0,
  );
  outputs=[:product1, :product2],
  output_spec=GEMB.CETSpec(
    [0.5, 0.5];
    et=2.0,
    alpha=1.0,
  ),
  demands=[:labor, :land],
  activity_start=100.0,
  name=:firm,
)

```

In this construction, the CES specification determines the inputs required for a given activity level, while the CET specification determines how that activity is transformed into the two outputs. If no `output_spec` is supplied, output quantities are instead proportional to the activity level through fixed output coefficients. Consequently, `output_spec` and fixed `output_coefficients` represent alternative ways of describing the output side and are not used simultaneously.

This separation between input and output specifications is particularly useful for joint production. The input side determines the resources required to support an activity level, while the output side determines the composition of the commodities supplied by that activity. The same activity variable therefore links production and transformation within a single economic agent.

4.2 Equivalent Representations Using Auxiliary Commodities and Virtual Producers

In addition to representing an economic structure directly through the specifications described above, GEMB models can sometimes be constructed more conveniently by introducing auxiliary commodities, additional factors, or virtual producers. Such transformations enlarge the formal representation of the model while preserving the underlying equilibrium relationships for the original commodities and economic agents.

1. A production function with nonconstant returns to scale can be represented as a constant-returns-to-scale technology by introducing an additional commodity with fixed aggregate supply. We refer to this commodity as a scale claim. This well-known **degree-one homogenization** method applies to decreasing, increasing, or variable returns to scale (see Section 5.4 for details). When the original production function exhibits decreasing returns to scale, the scale claim may also be interpreted as an entrepreneurial factor (McKenzie, 2002, p. 216).
2. Nested production or utility functions can be represented using virtual producers. Each nest is treated as a separate production activity whose output is an intermediate virtual commodity used by the next nest. In the case of consumer preferences, a consumer can equivalently be decomposed into a virtual utility producer and a utility consumer. The utility producer uses the consumer's ordinary commodities as inputs and produces a virtual utility commodity according to the original utility function, while the utility consumer consumes only this utility commodity.
3. Virtual producers can also be used to represent taxes imposed on only part of a producer's inputs. Taxed inputs can be distinguished from their corresponding untaxed commodities by introducing virtual taxed-input commodities. Virtual producers convert the original commodities into the corresponding taxed-input commodities while incorporating the relevant tax payments. The actual producer then demands these taxed-input commodities instead of the corresponding original commodities, whereas untaxed inputs continue to enter the production activity directly.

These constructions share a common modeling principle: auxiliary commodities, additional factors, or virtual producers are introduced to transform a nonstandard structure into an equivalent system of standard activities and market relationships.

4.3 CES-CET Production: A 4×2 Model

Assume the following:

1. The economy contains four commodities and two economic agents, a firm and a consumer. Commodities 1 and 2 are products, while commodities 3 and 4 are labor and land, respectively. Commodity 1 is taken as the numeraire commodity.
2. The firm uses labor and land to generate the activity level z according to the production function $z = x_3^{2/3} x_4^{1/3}$, where x_3 and x_4 denote labor and land inputs, respectively. The activity is transformed into the two products according to the CET transformation function $y_1^2 + 2y_2^2 = 3z^2$.
3. The consumer is endowed with $\omega > 0$ units of labor and $\omega' > 0$ units of land and consumes all four commodities. Its utility function is $u = c_1^{1/6} c_2^{1/3} c_3^{1/3} c_4^{1/6}$, where c_i denotes consumption of commodity i .

Let the price vector be $\mathbf{p} = (1, p_2, p_3, p_4)^\top$. The firm's conditional factor demand and product supply vectors are

$$\mathbf{x}(\mathbf{p}, z) = z \left(\left(\frac{2p_4}{p_3} \right)^{1/3}, \left(\frac{p_3}{2p_4} \right)^{2/3} \right)^\top$$

and

$$\mathbf{y}(\mathbf{p}, z) = \frac{3z}{(3 + \frac{3}{2}p_2^2)^{1/2}} \left(1, \frac{1}{2}p_2 \right)^\top$$

The consumer's income is $w = \omega p_3 + \omega' p_4$, and its demand vector is

$$\mathbf{c}(\mathbf{p}) = w \left(\frac{1}{6}, \frac{1}{3p_2}, \frac{1}{3p_3}, \frac{1}{6p_4} \right)^\top$$

Hence the demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, z) = \begin{bmatrix} 0 & c_1 \\ 0 & c_2 \\ x_3 & c_3 \\ x_4 & c_4 \end{bmatrix}$$

and

$$\mathbf{S}(\mathbf{p}, z) = \begin{bmatrix} y_1 & 0 \\ y_2 & 0 \\ 0 & \omega \\ 0 & \omega' \end{bmatrix}$$

Consider the following equality-form wide-sense structural equilibrium model:

$$\begin{aligned} \mathbf{p}^\top \mathbf{D}(\mathbf{p}, z) &= \mathbf{p}^\top \mathbf{S}(\mathbf{p}, z) \\ \mathbf{D}(\mathbf{p}, z) \mathbf{1} &= \mathbf{S}(\mathbf{p}, z) \mathbf{1} \end{aligned}$$

The equilibrium price vector and firm activity level are, respectively,

$$\mathbf{p}^* = \left(1, 2, 2 \left(\frac{\omega'}{\omega} \right)^{1/3}, \left(\frac{\omega}{\omega'} \right)^{2/3} \right)^\top$$

and

$$z^* = \frac{1}{2}\omega^{2/3}\omega'^{1/3}$$

The equilibrium product supply vector, factor demand vector, and consumer demand vector are

$$\mathbf{y}^* = z^*(1, 1)^\top$$

$$\mathbf{x}^* = \left(\frac{\omega}{2}, \frac{\omega'}{2}\right)^\top$$

and

$$\mathbf{c}^* = \left(z^*, z^*, \frac{\omega}{2}, \frac{\omega'}{2}\right)^\top$$

Hence the equilibrium demand and supply matrices are

$$\mathbf{D}^* = \begin{bmatrix} 0 & z^* \\ 0 & z^* \\ \frac{\omega}{2} & \frac{\omega}{2} \\ \frac{\omega'}{2} & \frac{\omega'}{2} \end{bmatrix}, \quad \mathbf{S}^* = \begin{bmatrix} z^* & 0 \\ z^* & 0 \\ 0 & \omega \\ 0 & \omega' \end{bmatrix}$$

The consumer's equilibrium utility level is

$$u^* = z^* = \frac{1}{2}\omega^{2/3}\omega'^{1/3}$$

In particular, when $\omega = \omega' = 100$,

$$\mathbf{p}^* = (1, 2, 2, 1)^\top, \quad z^* = 50, \quad u^* = 50$$

and

$$\mathbf{D}^* = \begin{bmatrix} 0 & 50 \\ 0 & 50 \\ 50 & 50 \\ 50 & 50 \end{bmatrix}, \quad \mathbf{S}^* = \begin{bmatrix} 50 & 0 \\ 50 & 0 \\ 0 & 100 \\ 0 & 100 \end{bmatrix}$$

Listing 4.1 A 4×2 Equilibrium Model with CES Inputs and CET Outputs

```
# =====
# gemb_CES_CET_jointProduction_nc4_na2_v1.jl
#
# CES-CET joint-production equilibrium:
# one producer and one consumer who consumes all four commodities.
#
# Commodities:
# 1. product1
# 2. product2
# 3. labor
# 4. land
# =====

using GeneralEquilibriumModeling

# -----
# 1. Parameters
# -----

omega_labor = 100.0
omega_land = 100.0

# Production:
#
#  $z = x_{\text{labor}}^{2/3} * x_{\text{land}}^{1/3}$ 
#
input_spec = GEMB.CESSpec(
    [2 / 3, 1 / 3];
    es=1.0,
    alpha=1.0,
)

# CET transformation:
#
#  $y_1^2 + 2*y_2^2 = 3*z^2$ 
#
output_spec = GEMB.CETSpec(
    [2 / 3, 1 / 3];
    et=1.0,
    alpha=sqrt(2) / 3,
)

# Consumer utility:
#
#  $u = c_1^{1/6} * c_2^{1/3} * c_3^{1/3} * c_4^{1/6}$ 
#
utility_spec = GEMB.CESSpec(
    [1 / 6, 1 / 3, 1 / 3, 1 / 6];
    es=1.0,
    alpha=1.0,
)
```

4.4 Ad Valorem Tax and Ad Valorem Claim: A 3×2 Model

4.4.1 Ad Valorem Claim

An ad valorem tax is levied as a fixed proportion of the value of a transaction, output, input, or other economic base. More generally, this concept can be extended to an ad valorem claim, which represents a right to claim a fixed proportion of a specified economic value. This concept provides a unified way to represent a variety of value-based claims in equilibrium models.

1. Let V denote the value base and τ the ad valorem claim rate. The value of the claim is τV . If the claim commodity has price p_c and quantity c , then $p_c c = \tau V$.
2. A positive claim rate, $\tau > 0$, can represent an ad valorem tax, fee, or other positive value claim, while a negative claim rate, $\tau < 0$, can represent an ad valorem subsidy or other reverse claim. Money and equity in equilibrium models can also be interpreted as forms of ad valorem claims.
3. By treating an ad valorem claim as a commodity and incorporating it into demand, supply, and revenue-expenditure balance, ad valorem taxes, subsidies, value-sharing arrangements, rent claims, money, equity, and related institutions can be represented within a unified wide-sense structural equilibrium framework.
4. Under an appropriate claim-based representation, profit rates, markup rates, saving rates, interest rates, rates of return, and ad valorem tax or subsidy rates can all be interpreted as ad valorem claim rates. In particular, money may serve as a claim-bearing commodity, while saving may be represented as the purchase of claims proportional to consumption expenditure.

4.4.2 A 3×2 Model with Ad Valorem Tax

Assume the following:

1. The economy contains three commodities: a product, labor, and a tax certificate. There are two economic agents: a firm and a consumer. Labor is taken as the numeraire commodity.
2. The firm produces and supplies the product according to the production function αx_2 , where x_2 denotes labor input and $\alpha > 0$.
3. The consumer has a labor endowment of ω and the utility function $x_1^\beta x_2^{1-\beta}$, where x_1 denotes the quantity of the product consumed and x_2 denotes the quantity of labor enjoyed by the consumer, or equivalently leisure, with $0 < \beta < 1$.

4. An ad valorem tax is imposed on the product, and the tax revenue is transferred to the consumer. The tax rate satisfies $0 < \tau < 1$, so the firm pays a tax equal to the fraction τ of its product sales revenue $p_1 y$. The consumer is endowed with one unit of the tax certificate.

Let $\mathbf{p} := (p_1, 1, p_3)^\top$, let y denote output, and let w denote consumer income. The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, w, y) := \begin{bmatrix} 0 & \frac{\beta w}{p_1} \\ \frac{y}{\alpha} & (1 - \beta)w \\ \frac{\tau p_1 y}{p_3} & 0 \end{bmatrix}$$

and

$$\mathbf{S}(y) := \begin{bmatrix} y & 0 \\ 0 & \omega \\ 0 & 1 \end{bmatrix}$$

Solving the equality-form wide-sense structural equilibrium model gives

$$\mathbf{p}^* = \left(\frac{1}{\alpha(1 - \tau)}, 1, \frac{\beta \omega \tau}{1 - \beta \tau} \right)^\top$$

$$w^* = \frac{\omega}{1 - \beta \tau}$$

and

$$y^* = \frac{\alpha \beta (1 - \tau) \omega}{1 - \beta \tau}$$

The consumer's equilibrium utility level is

$$u^* = \frac{\omega (\alpha \beta)^\beta (1 - \beta)^{1 - \beta} (1 - \tau)^\beta}{1 - \beta \tau}$$

For $0 < \tau < 1$,

$$\frac{\partial \ln u^*}{\partial \tau} = -\frac{\beta(1 - \beta)\tau}{(1 - \tau)(1 - \beta \tau)} < 0$$

Hence, the consumer's equilibrium utility decreases as the ad valorem tax rate increases. The tax therefore generates a deadweight loss, or excess burden, in this example.

For example, let $\alpha = 2$, $\beta = 1/2$, $\omega = 90$, and $\tau = 1/2$. The equilibrium is then particularly simple:

$$\mathbf{p}^* = (1, 1, 30)^\top, \quad w^* = 120, \quad y^* = 60$$

The consumer demands 60 units of the product and enjoys 60 units of labor, so the equilibrium utility level is $u^* = 60$. The corresponding demand and supply matrices are

$$\mathbf{D}^* = \begin{bmatrix} 0 & 60 \\ 30 & 60 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{S}^* = \begin{bmatrix} 60 & 0 \\ 0 & 90 \\ 0 & 1 \end{bmatrix}$$

Thus, all commodity markets clear exactly, and the revenue-expenditure balance condition is satisfied for both the firm and the consumer.

Listing 4.2 A 3×2 Equilibrium Model with an Ad Valorem Product Tax

```
using GeneralEquilibriumModeling

alpha = 2.0
beta = 0.5
omega = 90.0
tau = 0.5

claim_rate = tau / (1.0 - tau)

model = GEMB.GEMBModel(
    [:product, :labor, :tax_certificate];
    numeraire=:labor,
    numeraire_value=1.0,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec(
        [1.0];
        alpha=alpha,
    );
    outputs=:product,
    demands=:labor,
    claim_rate=claim_rate,
    claim=GEMB.CommodityRef(:tax_certificate),
    activity_start=100.0,
    name=:firm,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec(
        [beta, 1.0 - beta];
        es=1.0,
        alpha=1.0,
    );
    demands=[:product, :labor],
    endowments=[:labor, :tax_certificate],
    endowment_quantities=[omega, 1.0],
    activity_start=100.0,
    name=:consumer,
)

result = GEMB.solve(
    model;
    p0=[1.0, 1.0, 30.0],
    residual_tol=1.0e-8,
    silent=true,
)

stats = GEMB.equilibrium_statistics(model, result)
```

4.5 Specific Tax: A 3×2 Model

Consider the same economy as in the previous section, but replace the ad valorem tax with a specific tax on the product. Let $\tau > 0$ denote the tax per unit of output. The firm therefore pays τy in taxes when producing y units of the product, and the tax revenue is transferred to the consumer through the tax certificate.

Let $\mathbf{p} := (p_1, 1, p_3)^\top$, where labor is the numeraire commodity. The demand matrix becomes

$$\mathbf{D}(\mathbf{p}, w, y) = \begin{bmatrix} 0 & \frac{\beta w}{p_1} \\ \frac{y}{\alpha} & (1 - \beta)w \\ \frac{\tau y}{p_3} & 0 \end{bmatrix}$$

while the supply matrix remains

$$\mathbf{S}(y) = \begin{bmatrix} y & 0 \\ 0 & \omega \\ 0 & 1 \end{bmatrix}$$

Solving the equality-form wide-sense structural equilibrium model gives

$$\mathbf{p}^* = \left(\frac{1}{\alpha} + \tau, 1, \frac{\alpha\beta\omega\tau}{1 + \alpha\tau(1 - \beta)} \right)^\top$$

$$w^* = \frac{(1 + \alpha\tau)\omega}{1 + \alpha\tau(1 - \beta)}$$

and

$$y^* = \frac{\alpha\beta\omega}{1 + \alpha\tau(1 - \beta)}$$

The consumer's equilibrium utility level is

$$u^* = \frac{\omega(\alpha\beta)^\beta ((1 - \beta)(1 + \alpha\tau))^{1-\beta}}{1 + \alpha(1 - \beta)\tau}$$

For example, let $\alpha = 2$, $\beta = 1/2$, $\omega = 90$, and $\tau = 1/2$. The equilibrium is

$$\mathbf{p}^* = (1, 1, 30)^\top, \quad w^* = 120, \quad y^* = 60, \quad u^* = 60$$

For these parameter values, the numerical equilibrium happens to coincide with that of the ad valorem tax example in the previous section, although the definitions of the two tax rates are different.

Listing 4.3 A 3×2 Equilibrium Model with a Specific Tax

```
using GeneralEquilibriumModeling

alpha = 2.0
beta = 0.5
omega = 90.0
tau = 0.5

model = GEMB.GEMBModel(
    [:product, :labor, :tax_certificate];
    numeraire=:labor,
    numeraire_value=1.0,
)

firm_spec = GEMB.ActivityDemandSpec(
    function (output, prices)
        p_labor, p_tax = prices

        return [
            output / alpha,
            tau * output / p_tax,
        ]
    end,
)

GEMB.add_agent!(
    model,
    firm_spec;
    outputs=:product,
    demands=[:labor, :tax_certificate],
    activity_start=100.0,
    name=:firm,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec(
        [beta, 1.0 - beta];
        es=1.0,
        alpha=1.0,
    );
    demands=[:product, :labor],
    endowments=[:labor, :tax_certificate],
    endowment_quantities=[omega, 1.0],
    activity_start=100.0,
    name=:consumer,
)

result = GEMB.solve(
    model;
    p0=[1.0, 1.0, 30.0],
    residual_tol=1.0e-8,
```

4.6 Non-rival Service and Lindahl Prices: A 3×3 Model

Assume the following:

1. The economy contains three commodities: service 1, service 2, and labor. There are three economic agents: one firm and two consumers. Labor is taken as the numeraire commodity.
2. The firm provides a non-rival service, such as residential security service. Each unit of labor input provides one unit of personalized service to each consumer. These personalized services are referred to as service 1 and service 2, respectively.
3. Both consumers demand their personalized service and labor and have Cobb–Douglas utility functions. For consumer 1, the expenditure shares on service 1 and labor are β_1 and $1 - \beta_1$, respectively. For consumer 2, the corresponding shares on service 2 and labor are β_2 and $1 - \beta_2$.
4. Consumers 1 and 2 are endowed with $\omega_1 > 0$ and $\omega_2 > 0$ units of labor, respectively.

Let $\mathbf{p} := (p_1, p_2, 1)^\top$, let y denote the service level, and let the consumer-income vector be $\mathbf{w} = (w_1, w_2)^\top$. The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, \mathbf{w}, y) := \begin{bmatrix} 0 & \frac{\beta_1 w_1}{p_1} & 0 \\ 0 & 0 & \frac{\beta_2 w_2}{p_2} \\ y & (1 - \beta_1)w_1 & (1 - \beta_2)w_2 \end{bmatrix}$$

and

$$\mathbf{S}(y) := \begin{bmatrix} y & 0 & 0 \\ y & 0 & 0 \\ 0 & \omega_1 & \omega_2 \end{bmatrix}$$

Solving the equality-form wide-sense structural equilibrium model gives

$$\mathbf{p}^* = \left(\frac{\beta_1 \omega_1}{\beta_1 \omega_1 + \beta_2 \omega_2}, \frac{\beta_2 \omega_2}{\beta_1 \omega_1 + \beta_2 \omega_2}, 1 \right)^\top$$

$$\mathbf{w}^* = (\omega_1, \omega_2)^\top$$

and

$$y^* = \beta_1 \omega_1 + \beta_2 \omega_2$$

The personalized equilibrium prices satisfy

$$p_1^* + p_2^* = 1$$

so the sum of the two consumers' personalized prices equals the marginal cost of providing the non-rival service. The prices p_1^* and p_2^* can therefore be interpreted as Lindahl prices: each consumer faces a personalized price for the same underlying non-rival service, while the sum of these prices covers its unit production cost.

For example, let $\beta_1 = 1/4$, $\beta_2 = 2/3$, $\omega_1 = 120$, and $\omega_2 = 90$. The equilibrium is

$$\mathbf{p}^* = \left(\frac{1}{3}, \frac{2}{3}, 1\right)^\top, \quad \mathbf{w}^* = (120, 90)^\top, \quad y^* = 90$$

Both consumers receive 90 units of the non-rival service. Consumer 1 enjoys 90 units of labor, whereas consumer 2 enjoys 30 units. The corresponding demand and supply matrices are

$$\mathbf{D}^* = \begin{bmatrix} 0 & 90 & 0 \\ 0 & 0 & 90 \\ 90 & 90 & 30 \end{bmatrix}, \quad \mathbf{S}^* = \begin{bmatrix} 90 & 0 & 0 \\ 90 & 0 & 0 \\ 0 & 120 & 90 \end{bmatrix}$$

The two personalized Lindahl prices are therefore $1/3$ and $2/3$, reflecting the consumers' different valuations of the non-rival service, while their sum remains equal to the unit cost of providing it.

Listing 4.4 A 3×3 Equilibrium Model with a Non-rival Service and Lindahl Prices

```
using GeneralEquilibriumModeling

beta1 = 1 / 4
beta2 = 2 / 3
omega1 = 120.0
omega2 = 90.0

model = GEMB.GEMBModel(
    [:service1, :service2, :labor];
    numeraire=:labor,
    numeraire_value=1.0,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec([1.0]);
    outputs=[:service1, :service2],
    demands=:labor,
    activity_start=100.0,
    name=:firm,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec([beta1, 1 - beta1]);
    demands=[:service1, :labor],
    endowments=:labor,
    endowment_quantities=omega1,
    activity_start=100.0,
    name=:consumer1,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec([beta2, 1 - beta2]);
    demands=[:service2, :labor],
    endowments=:labor,
    endowment_quantities=omega2,
    activity_start=100.0,
    name=:consumer2,
)

result = GEMB.solve(
    model;
    p0=[0.5, 0.5, 1.0],
    residual_tol=1.0e-8,
    silent=true,
)

GEMB.print_equilibrium_statistics(model, result)
```

4.7 Capital Rental and Equity: A 4×3 Model

Assume the following (see Torres, 2016, Tables 2.1 and 2.2):

1. The economy contains four commodities: a product, labor, capital services, and equity. There are three economic agents: a production firm, a consumer, and a capital-rental firm. Equity, money, and ad valorem tax certificates can all be regarded as ad valorem claims.
2. The production firm produces the product according to $y = x_2^{1-\beta_1} x_3^{\beta_1}$, where x_2 and x_3 denote labor and capital-service inputs, respectively, with $0 < \beta_1 < 1$.
3. The consumer has utility function $u = x_1^{\beta_2} x_2^{1-\beta_2}$, where $0 < \beta_2 < 1$, and is endowed with one unit of labor and one unit of equity.
4. The capital-rental firm uses one unit of the product to provide one unit of capital services and recover $1 - \delta$ units of the product after depreciation. Let $r > 0$ denote the required rate of return. For each unit of activity, the firm demands equity whose value is rp_1 .

Let $\mathbf{p} = (p_1, p_2, p_3, p_4)^\top$, and let y , u , and k denote the activity levels of the production firm, the consumer, and the capital-rental firm, respectively. The demand matrix is

$$\mathbf{D}(\mathbf{p}, y, u, k) = \begin{bmatrix} 0 & u \left(\frac{\beta_2}{1-\beta_2} \frac{p_2}{p_1} \right)^{1-\beta_2} & k \\ y \left(\frac{1-\beta_1}{\beta_1} \frac{p_3}{p_2} \right)^{\beta_1} & u \left(\frac{1-\beta_2}{\beta_2} \frac{p_1}{p_2} \right)^{\beta_2} & 0 \\ y \left(\frac{\beta_1}{1-\beta_1} \frac{p_2}{p_3} \right)^{1-\beta_1} & 0 & 0 \\ 0 & 0 & \frac{rp_1}{p_4} k \end{bmatrix}$$

and the supply matrix is

$$\mathbf{S}(y, k) = \begin{bmatrix} y & 0 & (1-\delta)k \\ 0 & 1 & 0 \\ 0 & 0 & k \\ 0 & 1 & 0 \end{bmatrix}$$

By solving the wide-sense structural equilibrium model, the analytical solution for the equilibrium prices can be obtained. Taking the product as the numeraire commodity, so that $p_1^* = 1$, the analytical equilibrium prices are

$$p_3^* = r + \delta, \quad p_2^* = (1 - \beta_1) \left(\frac{\beta_1}{r + \delta} \right)^{\frac{\beta_1}{1-\beta_1}}$$

and

$$p_4^* = \frac{r\beta_1\beta_2}{r(1-\beta_1\beta_2) + \delta(1-\beta_1)}p_2^*$$

For $r = 1/0.97 - 1 \approx 0.030928$, $\delta = 0.06$, $\beta_1 = 0.35$, and $\beta_2 = 0.4$, the equilibrium price vector is

$$\mathbf{p}^* \approx (1, 1.343115, 0.090928, 0.088654)^\top$$

Listing 4.5 A 4×3 Equilibrium Model with Capital Rental and Equity Claims

```
using GeneralEquilibriumModeling

discount_factor = 0.97
return_rate = 1 / discount_factor - 1
delta = 0.06
beta_firm = 0.35
beta_consumer = 0.4

model = GEMB.GEMBModel(
    [
        :product,
        :labor,
        :capital_service,
        GEMB.CommoditySpec(:equity; price_lower_bound=1.0e-10),
    ];
    numeraire=:product,
    numeraire_value=1.0,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec([1 - beta_firm, beta_firm]);
    outputs=:product,
    demands=[:labor, :capital_service],
    activity_start=100.0,
    name=:production_firm,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec([beta_consumer, 1 - beta_consumer]);
    demands=[:product, :labor],
    endowments=[:labor, :equity],
    endowment_quantities=[1.0, 1.0],
    activity_start=100.0,
    name=:consumer,
)

GEMB.add_agent!(
    model,
    GEMB.CESSpec([1.0]);
    outputs=[:product, :capital_service],
    output_coefficients=[1 - delta, 1.0],
    demands=:product,
    claim_rate=return_rate,
    claim=GEMB.CommodityRef(:equity),
    activity_start=100.0,
    name=:capital_rental_firm,
)

result = GEMB.solve(
```

5 Static Equilibrium with Production: Extended Models

5.1 Pollution and Negative Prices: A 3×2 Model

Assume the following:

1. The economy contains three commodities and two economic agents, a firm and a consumer. Commodities 1, 2, and 3 are a product, labor, and pollution, respectively. Commodity 2 is taken as the numeraire commodity, so $p_2 = 1$.
2. The firm operates at activity level z . Each unit of activity uses $a > 0$ units of labor, produces one unit of the product, and jointly generates $c > 0$ units of pollution. The product output per unit of activity is normalized to one.
3. The consumer is endowed with $\omega > 0$ units of labor and consumes x units of the product, retains l units of labor, and accepts q units of pollution. Its utility function is $u = \beta \ln x + (1 - \beta) \ln l - \delta q$, where $0 < \beta < 1$ and $\delta > 0$.

For a given price vector $\mathbf{p} = (p_1, 1, p_3)^\top$, the consumer's optimal demands are

$$x(\mathbf{p}) = -\frac{\beta p_3}{\delta p_1}, \quad l(\mathbf{p}) = -\frac{(1 - \beta)p_3}{\delta}, \quad q(\mathbf{p}) = \frac{1}{\delta} + \frac{\omega}{p_3}$$

for an interior solution with $p_3 < -\delta\omega < 0$. Hence the demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, z) = \begin{bmatrix} 0 & -\frac{\beta p_3}{\delta p_1} \\ az & -\frac{(1 - \beta)p_3}{\delta} \\ 0 & \frac{1}{\delta} + \frac{\omega}{p_3} \end{bmatrix}, \quad \mathbf{S}(z) = \begin{bmatrix} z & 0 \\ 0 & \omega \\ cz & 0 \end{bmatrix}$$

The equilibrium can then be obtained from the equality-form wide-sense structural equilibrium model.

Define $\Delta = a + c\delta\omega$. The unique economically relevant equilibrium activity level is

$$z^* = \frac{\Delta - \sqrt{\Delta^2 - 4ac\delta\beta\omega}}{2ac\delta}$$

with $0 < z^* < \min \omega/a, \beta/(c\delta)$. The equilibrium quantities are $x^* = z^*$, $l^* = \omega - az^*$, and $q^* = cz^*$, while

$$p_1^* = \frac{\beta(\omega - az^*)}{(1 - \beta)z^*}, \quad p_2^* = 1, \quad p_3^* = -\frac{\delta(\omega - az^*)}{1 - \beta}, \quad \lambda^* = \frac{1 - \beta}{\omega - az^*}$$

Thus, a commodity need not have a nonnegative equilibrium price in a wide-sense structural equilibrium model. When a commodity is an undesirable joint product such as pollution, its negative price represents the compensation required for another economic agent to accept it.

As a simple numerical case, let $a = c = 1$, $\beta = 1/2$, $\delta = 1/8$, and $\omega = 6$. Then $z^* = 2$, $(x^*, l^*, q^*) = (2, 4, 2)$, and

$$\mathbf{p}^* = [2, 1, -1]^\top$$

so the equilibrium price of pollution is $p_3^* = -1$.

Listing 5.1 A 3×2 Equilibrium Model with Pollution and a Negative Price

```
# GEMB high-level framework: pollution with a negative equilibrium price.
using GeneralEquilibriumModeling

a = 1.0
c = 1.0
beta = 0.5
delta = 1.0 / 8.0
omega = 6.0

model = GEMB.GEMBModel(
    [
        :product,
        :labor,
        GEMB.CommoditySpec(
            :pollution;
            price_lower_bound=-Inf,
            price_upper_bound=Inf,
        ),
    ];
    numeraire=:labor,
    numeraire_value=1.0,
)

firm_spec = GEMB.ActivityDemandSpec(
    (z, prices) -> [a * z],
)

GEMB.add_agent!(
    model,
    firm_spec;
    outputs=[:product, :pollution],
    output_coefficients=[1.0, c],
    demands=:labor,
    activity_start=100.0,
    name=:firm,
)

consumer_spec = GEMB.MarginalUtilityConsumerSpec(
    demand -> begin
        x, l, q = demand
        [
            beta / x,
            (1.0 - beta) / l,
            -delta,
        ]
    end,
)

GEMB.add_agent!(
    model,
    consumer_spec;
```

5.2 Producer-to-Consumer Externality: A 2×2 Model

Assume the following:

1. The economy contains two commodities and two economic agents, a firm and a consumer. Commodities 1 and 2 are a product and labor, respectively. Commodity 2 is taken as the numeraire commodity, so $p_2 = 1$.
2. The firm uses one unit of labor to produce one unit of the product. Let y denote its output.
3. The consumer is endowed with $\omega > 0$ units of labor and has utility function $u = (1 - \kappa y) \ln x_1 + \ln x_2$, where $\kappa > 0$ measures the negative external effect of the firm's production and $1 - \kappa y > 0$. The consumer takes y as given when choosing consumption.

For a given $\mathbf{p} = (p_1, p_2)^\top$ and y , the consumer's demands are

$$x_1(\mathbf{p}, y) = \frac{1 - \kappa y}{2 - \kappa y} \frac{\omega p_2}{p_1}, \quad x_2(\mathbf{p}, y) = \frac{\omega}{2 - \kappa y}$$

Hence the demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, y) = \begin{bmatrix} 0 & \frac{1 - \kappa y}{2 - \kappa y} \frac{\omega p_2}{p_1} \\ y & \frac{\omega}{2 - \kappa y} \end{bmatrix}, \quad \mathbf{S}(y) = \begin{bmatrix} y & 0 \\ 0 & \omega \end{bmatrix}$$

Applying the equality-form wide-sense structural equilibrium model yields $p_1^* = 1$ and

$$y^* = \frac{2 + \omega\kappa - \sqrt{4 + \omega^2\kappa^2}}{2\kappa}$$

The equilibrium consumption vector is $[x_1^*, x_2^*]^\top = [y^*, \omega - y^*]^\top$.

For $\omega = 12$ and $\kappa = 1/8$,

$$y^* = 4, \quad \mathbf{p}^* = [1, 1]^\top$$

Listing 5.2 A 2×2 Equilibrium Model with Producer-to-Consumer Externality

```
# GEMB high-level framework: production externality.
using GeneralEquilibriumModeling.GEMB

omega = 12.0
kappa = 1.0 / 8.0

model = GEMBModel(
    [:product, :labor];
    numeraire=:labor,
)

add_agent!(
    model,
    CESSpec([1.0]);
    outputs=:product,
    demands=:labor,
    activity_start=4.0,
    name=:firm,
)

add_agent!(
    model,
    MarshallDemandConsumerSpec(
        (income, prices, observed) -> begin
            activity = observed[1]
            [
                (1.0 - kappa * activity) / (2.0 - kappa * activity) *
                    income / prices[1],
                income / ((2.0 - kappa * activity) * prices[2]),
            ]
        end,
    );
    demands=:product, :labor,
    endowments=:labor,
    endowment_quantities=omega,
    observed_variables=[
        agent_variable_ref(AgentRef(:firm), :activity),
    ],
    name=:consumer,
)

result = solve(
    model;
    p0=[2.0, 1.0],
    silent=true,
)

print_equilibrium_statistics(model, result)
```

5.3 Marginal Pricing Principle and Scale Claim Rates

5.3.1 Scale Claim Rates

As noted earlier, profit rates, ad valorem subsidy rates, and similar value ratios can generally be interpreted as ad valorem claim rates.

A marginal pricing equilibrium (also called marginal cost pricing equilibrium) can be viewed as a natural extension of competitive equilibrium to nonconvex economies (Bonnisseau and Cornet, 1990; Quinzii, 1992). Under appropriate conditions, a Pareto-optimal allocation can often be represented as a marginal-pricing equilibrium allocation.

In a marginal-pricing equilibrium, an appropriate claim rate may be required to reconcile marginal pricing with the firm's revenue-expenditure balance. When this claim rate is determined by the firm's scale elasticity, it is referred to as the **scale claim rate**. The scale claim rate denotes the value of the scale claim relative to the value of ordinary inputs. When positive, it can be interpreted as a cost-based profit rate, or scale profit rate; when negative, its absolute value can be interpreted as a subsidy rate.

1. Let a firm's production function be $y = f(\mathbf{x})$. Its scale elasticity is

$$\varepsilon = \frac{\mathbf{x}^\top \nabla f(\mathbf{x})}{f(\mathbf{x})}$$

and the corresponding scale claim rate is

$$\tau_s = \frac{f(\mathbf{x})}{\mathbf{x}^\top \nabla f(\mathbf{x})} - 1 = \frac{1}{\varepsilon} - 1$$

2. Under marginal pricing, inputs are valued according to their marginal products. The scale claim rate measures the proportional claim required to reconcile the value of output with the value of inputs evaluated at marginal products. Thus, incorporating a claim at rate τ_s allows marginal pricing to be represented together with the firm's revenue-expenditure balance. The marginal-pricing principle can be applied not only to firms but also to consumer decision-making.
3. If $f(\mathbf{x})$ is homogeneous of degree $\theta > 0$, Euler's theorem gives $\mathbf{x}^\top \nabla f(\mathbf{x}) = \theta f(\mathbf{x})$, and hence

$$\tau_s = \frac{1}{\theta} - 1$$

Thus, $\tau_s = 0$ under constant returns to scale, $\tau_s > 0$ under decreasing returns to scale, and $-1 < \tau_s < 0$ under increasing returns to scale. A positive scale claim rate implies that the

firm distributes profits to its shareholders, whereas a negative scale claim rate implies that a subsidy must be provided to the firm to support marginal pricing.

5.3.2 Marginal Pricing with a Power Production Function: A 3×2 Model

Consider a simple equilibrium model with marginal pricing and an ad valorem claim. Assume the following:

1. The economy contains three commodities: a product, labor, and an ad valorem claim commodity. There is one firm and one consumer. Labor is the numeraire commodity.
2. The firm has the production function $y = \alpha x_2^\theta$, where x_2 is labor input and $\alpha, \theta > 0$.
3. The consumer has utility function $u = x_1^\beta x_2^{1-\beta}$, where $0 < \beta < 1$, and is endowed with $\omega > 0$ units of labor.
4. The consumer owns one unit of the claim commodity. A claim at rate $\tau > -1$ is imposed on the firm's input expenditure. A positive τ can be interpreted as profit distributed by the firm to the claim holder, whereas a negative τ represents a subsidy provided to the firm. The scale-claim representation below assumes $\tau \neq 0$. When $\tau = 0$, the scale claim has zero value and can simply be omitted from the model.

Let $\mathbf{p} = (p_1, 1, p_3)^\top$ and let w denote consumer income. The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, w, y) = \begin{bmatrix} 0 & \beta w / p_1 \\ (y/\alpha)^{1/\theta} & (1-\beta)w \\ (y/\alpha)^{1/\theta} \tau / p_3 & 0 \end{bmatrix}, \quad \mathbf{S}(y) = \begin{bmatrix} y & 0 \\ 0 & \omega \\ 0 & 1 \end{bmatrix}$$

Applying the equality-form wide-sense structural equilibrium model gives

$$p_1^* = \frac{1+\tau}{\alpha} \left(\frac{\beta\omega}{1+(1-\beta)\tau} \right)^{1-\theta}, \quad p_3^* = \frac{\tau\beta\omega}{1+(1-\beta)\tau}$$

and

$$w^* = \frac{\omega(1+\tau)}{1+(1-\beta)\tau}, \quad y^* = \alpha \left(\frac{\beta\omega}{1+(1-\beta)\tau} \right)^\theta$$

Under marginal pricing, the claim rate should satisfy the scale-claim-rate consistency condition

$$\tau = \frac{1}{\theta} - 1$$

The equilibrium then becomes

$$p_1^* = \frac{1}{\alpha\theta} \left(\frac{\beta\theta\omega}{1-\beta+\beta\theta} \right)^{1-\theta}, \quad p_3^* = \frac{\beta\omega(1-\theta)}{1-\beta+\beta\theta}$$

and

$$w^* = \frac{\omega}{1-\beta+\beta\theta}, \quad y^* = \alpha \left(\frac{\beta\theta\omega}{1-\beta+\beta\theta} \right)^\theta$$

Thus, the scale claim reconciles marginal pricing with the firm's revenue-expenditure balance.

For a simple comparison, let $\beta = 1/2$ and $\omega = 3$ in both cases. Under decreasing returns to scale, take $\theta = 1/2$ and $\alpha = 2$. The scale claim rate is then $\tau = 1$, and the equilibrium is $\mathbf{p}^* = [1, 1, 1]^\top$, $w^* = 4$, and $y^* = 2$. Under increasing returns to scale, take $\theta = 2$ and $\alpha = 1/4$. The corresponding scale claim rate is $\tau = -1/2$, and the equilibrium is $\mathbf{p}^* = [1, 1, -1]^\top$, $w^* = 2$, and $y^* = 1$.

Listing 5.3 A 3×2 Equilibrium Model with a Power Production Function and a Scale Claim

```
using GeneralEquilibriumModeling.GEMB

beta = 0.5
omega = 3.0

increasing_returns = true # Set to false for the alternative numerical example.

alpha = increasing_returns ? 0.25 : 2.0
theta = increasing_returns ? 2.0 : 0.5
tau = 1.0 / theta - 1.0

model = GEMBModel(
    [
        CommoditySpec(
            :product;
            price_lower_bound=1e-3,
        ),
        :labor,
        CommoditySpec(
            :claim;
            price_lower_bound=increasing_returns ? -Inf : 1e-3,
            price_upper_bound=increasing_returns ? 1e-3 : Inf,
        ),
    ];
    numeraire=:labor,
)

add_agent!(
    model,
    PowerProductionSpec(
        alpha=alpha,
        theta=theta,
    );
    outputs=:product,
    demands=:labor,
    claim=:claim,
    claim_rate=tau,
    activity_start=increasing_returns ? 0.8 : 1.5,
    activity_lower_bound=1.0e-6,
    name=:firm,
)

add_agent!(
    model,
    CESSpec([beta, 1.0 - beta]);
    demands=[:product, :labor],
    endowments=[:labor, :claim],
    endowment_quantities=[omega, 1.0],
    activity_start=1.5,
    name=:consumer,
)
```

5.3.3 Marginal Pricing with an Exponential Production Function: A 3×2 Model

Assume the following:

1. The economy contains three commodities and two economic agents, a firm and a consumer. Commodities 1, 2, and 3 are a product, labor, and a scale claim, respectively. Commodity 2 is taken as the numeraire commodity, so $p_2 = 1$.
2. The firm minimizes cost for a given output y and has the piecewise production function

$$f(x) = \begin{cases} \frac{\alpha\theta^\xi}{\xi}x, & 0 \leq x < \xi \\ \alpha\theta^x, & x \geq \xi \end{cases}$$

where $\alpha > 0$, $\theta > 1$, and $\xi > 0$. Only equilibria with $x^* > \xi$ are considered. In this region, $y = \alpha\theta^x$ and $x = \ln(y/\alpha)/\ln \theta$.

3. The consumer has utility function $u = x_1^\beta x_2^{1-\beta}$, where $0 < \beta < 1$, and is endowed with $\omega > 0$ units of labor and one unit of the scale claim.
4. Under marginal pricing, the scale claim rate depends on the firm's input and is given by

$$\tau(x) = \frac{1}{x \ln \theta} - 1$$

The scale-claim representation below assumes $\tau(x^*) \neq 0$. If $\tau(x^*) = 0$, the scale claim has zero value and can simply be omitted from the model.

Thus, unlike the power-production example, the scale claim rate is endogenous and varies with the equilibrium scale of production.

Let $\mathbf{p} = (p_1, 1, p_3)^\top$ and let w denote consumer income. The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, w, y) = \begin{bmatrix} 0 & \beta w/p_1 \\ x & (1-\beta)w \\ x\tau(x)/p_3 & 0 \end{bmatrix}, \quad \mathbf{S}(y) = \begin{bmatrix} y & 0 \\ 0 & \omega \\ 0 & 1 \end{bmatrix}, \quad x = \frac{\ln(y/\alpha)}{\ln \theta}$$

Applying the equality-form wide-sense structural equilibrium model together with the scale-claim-rate consistency condition yields

$$x^* = \omega - \frac{1-\beta}{\beta \ln \theta}$$

which requires

$$\omega > \xi + \frac{1 - \beta}{\beta \ln \theta}$$

The corresponding scale claim rate is

$$\tau^* = \frac{1 - \beta \omega \ln \theta}{\beta \omega \ln \theta - (1 - \beta)}$$

and the remaining equilibrium values are

$$p_1^* = \frac{e^{(1-\beta)/\beta}}{\alpha(\ln \theta)\theta^\omega}, \quad p_3^* = \frac{1}{\beta \ln \theta} - \omega$$

$$w^* = \frac{1}{\beta \ln \theta}, \quad y^* = \alpha \theta^\omega e^{-(1-\beta)/\beta}$$

Thus, the scale claim rate is determined jointly with the firm's production scale.

For a simple comparison, let $\beta = 1/2$, $\theta = e$, and $\xi = 1/4$ in both cases. When $\omega = 3/2$ and $\alpha = e^{-1/2}$, the equilibrium input is $x^* = 1/2$, the endogenous scale claim rate is $\tau^* = 1$, and the equilibrium is $\mathbf{p}^* = [1, 1, 1/2]^\top$, $w^* = 2$, and $y^* = 1$. When $\omega = 3$ and $\alpha = e^{-2}$, the equilibrium input is $x^* = 2$, the endogenous scale claim rate is $\tau^* = -1/2$, and the equilibrium is $\mathbf{p}^* = [1, 1, -1]^\top$, $w^* = 2$, and $y^* = 1$.

In the GEMB implementation, an additional condition agent is introduced to impose the endogenous scale-claim-rate consistency condition, so the computational model contains three agents although the economy itself contains only two economic agents.

Listing 5.4 An Equilibrium Model with Exponential Production and an Endogenous Scale Claim Rate

```

using GeneralEquilibriumModeling.GEMB

beta = 0.5
theta = exp(1.0)
xi = 0.25

positive_claim = false # Set to true for the alternative numerical example.

omega = positive_claim ? 1.5 : 3.0
alpha = positive_claim ? exp(-0.5) : exp(-2.0)

model = GEMBModel(
    [
        CommoditySpec(
            :product;
            price_lower_bound=1e-3,
        ),
        :labor,
        CommoditySpec(
            :scale_claim;
            price_lower_bound=positive_claim ? 1e-3 : -Inf,
            price_upper_bound=positive_claim ? Inf : -1e-3,
        ),
    ];
    numeraire=:labor,
)

add_agent!(
    model,
    ConditionAgentSpec(
        (variables, observed) -> begin
            tau = variables[1]
            activity = observed[1]

            [
                (1.0 + tau) * log(activity / alpha) - 1.0,
            ]
        end,
    );
    variable_names=:claim_rate,
    variable_start=positive_claim ? 0.6 : -0.3,
    variable_lower_bounds=-0.99,
    observed_variables=[
        agent_variable_ref(:firm, :activity),
    ],
    name=:scaleClaimSetter,
)

add_agent!(
    model,
    ConditionAgentSpec(

```

5.4 Degree-One Homogenization of Production Functions

5.4.1 Degree-One Homogenization and the Scale Claim Rate

Degree-one homogenization is commonly used to transform a production function with decreasing returns to scale into one with constant returns to scale (McKenzie, 2002, p. 216). More generally, the method can also be applied to production functions with increasing or variable returns to scale, thereby providing a unified constant-returns-to-scale representation for different types of returns to scale.

Consider a firm with production function $y = f(\mathbf{x})$, where $\mathbf{x} = (x_1, \dots, x_n)^\top$ denotes ordinary production inputs. The firm is assumed to follow the marginal-pricing principle, represented in GEMB by the first-order stationarity conditions with `behavior=:stationary`.

Introduce an additional commodity, called a scale claim, with quantity $c > 0$, and define the degree-one homogenized production function

$$g(\mathbf{x}, c) = cf\left(\frac{\mathbf{x}}{c}\right)$$

where $\mathbf{x}/c = (x_1/c, \dots, x_n/c)^\top$. The function $g(\mathbf{x}, c)$ is homogeneous of degree one in $(\mathbf{x}, c)$. Suppose the aggregate supply of the scale claim is normalized to one. Market clearing then implies $c = 1$.

Let p_y denote the output price, $\mathbf{p}_x = (p_{x1}, \dots, p_{xn})^\top$ the prices of ordinary inputs, and p_c the price of the scale claim. Under marginal pricing, the stationarity conditions at $c = 1$ imply

$$\mathbf{p}_x = p_y \nabla f(\mathbf{x})$$

and

$$p_c = p_y (f(\mathbf{x}) - \mathbf{x}^\top \nabla f(\mathbf{x}))$$

Since g is homogeneous of degree one, the firm's revenue-expenditure balance is

$$p_y y = \mathbf{p}_x^\top \mathbf{x} + p_c c$$

Define the scale claim rate as the ratio of the value of the scale claim to the value of ordinary inputs. Then

$$\tau(\mathbf{x}) = \frac{f(\mathbf{x})}{\mathbf{x}^\top \nabla f(\mathbf{x})} - 1$$

Let the local scale elasticity of production be

$$\varepsilon_f(\mathbf{x}) = \frac{\mathbf{x}^\top \nabla f(\mathbf{x})}{f(\mathbf{x})}$$

The scale claim rate can therefore be written as

$$\tau(\mathbf{x}) = \frac{1}{\varepsilon_f(\mathbf{x})} - 1$$

Thus, locally decreasing returns to scale imply a positive scale claim rate, constant returns to scale imply a zero rate, and locally increasing returns to scale imply a negative rate.

5.4.2 Equivalence under Degree-One Homogenization

Degree-one homogenization expands the commodity space by introducing the scale claim while preserving the original production technology when $c = 1$. In particular,

$$g(\mathbf{x}, 1) = f(\mathbf{x})$$

and

$$\nabla_{\mathbf{x}} g(\mathbf{x}, 1) = \nabla f(\mathbf{x})$$

Therefore, when the scale-claim market clears at $c = 1$, the technological relation between the original inputs and output is unchanged, and the marginal-pricing conditions for the ordinary inputs are identical to those in the original model. The additional stationarity condition associated with the scale claim implies

$$p_c = p_y f(\mathbf{x}) - \mathbf{p}_x^\top \mathbf{x}$$

Thus, the difference between output value and the value of ordinary inputs under marginal pricing is represented as the value of the scale claim. The scale-claim price p_c is an endogenous equilibrium price determined jointly with the other prices and quantities.

Hence, on the production side, degree-one homogenization provides an equivalent representation of the original technology and its marginal-pricing conditions in an expanded commodity space. Equivalence of the complete equilibrium system additionally requires consistency in the treatment of the scale claim. Its aggregate supply must be normalized so that market clearing implies $c = 1$, and its endowment must be allocated so that the resulting scale-claim value corresponds to the distribution of the residual between output value and ordinary-input value

in the original model. All other endowments, technologies, preferences, and market-clearing conditions remain unchanged.

5.4.3 Power and Exponential Production Functions

For the power production function

$$f(x) = \alpha x^\theta$$

degree-one homogenization gives

$$g(x, c) = cf\left(\frac{x}{c}\right) = \alpha x^\theta c^{1-\theta}$$

For the exponential branch of the production function,

$$f(x) = \alpha \theta^x, \quad x \geq \xi > 0$$

degree-one homogenization gives

$$g(x, c) = cf\left(\frac{x}{c}\right) = \alpha c \theta^{x/c}, \quad \frac{x}{c} \geq \xi > 0, \quad c > 0$$

In GEMB, the scale claim commodity can be included as an additional production input in a `ProductionFunctionSpec`, with its aggregate supply normalized to one and the firm specified with `behavior=:stationary`. When the marginal products contain a common strictly positive factor, this factor may be omitted to obtain a scaled marginal-product function. Such scaling changes only the production multiplier and leaves equilibrium quantities and prices unchanged.

The power production case is implemented in Listing [5.5](#).

Listing 5.5 Degree-One Homogenization of a Power Production Function

```
using GeneralEquilibriumModeling

decreasing_returns = false # Set to true for the alternative numerical example.

beta = 0.5
omega = 3.0

alpha = decreasing_returns ? 2.0 : 0.25
theta = decreasing_returns ? 0.5 : 2.0
tau = 1.0 / theta - 1.0

production_function = inputs -> begin
    x, c = inputs
    alpha * x^theta * c^(1.0 - theta)
end

marginal_product_function = inputs -> begin
    x, c = inputs

    [
        alpha * theta * x^(theta - 1.0) * c^(1.0 - theta),
        alpha * (1.0 - theta) * x^theta * c^(-theta),
    ]
end

model = GEMB.GEMBModel(
    [
        :product,
        GEMB.CommoditySpec(:labor; price_lower_bound=1.0e-10),
        GEMB.CommoditySpec(
            :scale_claim;
            price_lower_bound=-Inf,
            price_upper_bound=Inf,
        ),
    ];
    numeraire=:product,
    numeraire_value=1.0,
)

GEMB.add_agent!(
    model,
    GEMB.ProductionFunctionSpec(
        production_function,
        marginal_product_function;
        behavior=:stationary,
    );
    outputs=:product,
    demands=[:labor, :scale_claim],
    activity_start=decreasing_returns ? 2.0 : 1.0,
    demand_start=[decreasing_returns ? 1.0 : 2.0, 1.0],
    demand_lower_bounds=[1.0e-10, 1.0e-10],
    106
```

The exponential production case illustrates a production function whose scale elasticity, and hence scale claim rate, varies with the input level. The corresponding implementation is given in Listing [5.6](#).

Listing 5.6 Degree-One Homogenization of an Exponential Production Function

```
using GeneralEquilibriumModeling

decreasing_returns = false # Set to true for the alternative numerical example.

beta = 0.5
omega = 3.0

theta = decreasing_returns ? exp(0.5) : exp(1.0)
alpha = decreasing_returns ? 2.0 / theta : 1.0 / theta^2

output_start = 1.5
labor_start = 1.5
scale_claim_start = 1.2

p0 = decreasing_returns ?
    [1.0, 0.8, 0.5] :
    [1.0, 0.8, -0.5]

# Degree-one homogenization:
#  $g(x, c) = c * f(x / c) = \alpha * c * \theta^{(x / c)}$ 
production_function = inputs -> begin
    x, c = inputs
    alpha * c * theta^(x / c)
end

# Scaled marginal product function:
# the common strictly positive factor  $\alpha * \theta^{(x / c)}$  is omitted.
marginal_product_function = inputs -> begin
    x, c = inputs

    [
        log(theta),
        1.0 - (x / c) * log(theta),
    ]
end

model = GEMB.GEMBModel(
    [
        :product,
        GEMB.CommoditySpec(
            :labor;
            price_lower_bound=1.0e-10,
        ),
        GEMB.CommoditySpec(
            :scale_claim;
            price_lower_bound=-Inf,
            price_upper_bound=Inf,
        ),
    ];
    numeraire=:product,
    numeraire_value=1.0,
```

5.4.4 Square-Sum Production Function and Marginal Pricing Equilibria: A 4×2 Model

Assume the following:

1. The economy contains four commodities: a product, labor, land, and a scale claim. There are two economic agents, a firm and a consumer. The product is taken as the numeraire commodity.
2. The firm's original production function is $f(x_2, x_3) = x_2^2 + x_3^2$, where x_2 and x_3 denote labor and land inputs, respectively. Introduce the scale claim input $x_4 > 0$ and degree-one homogenize the production function as

$$g(x_2, x_3, x_4) = x_4 f\left(\frac{x_2}{x_4}, \frac{x_3}{x_4}\right) = \frac{x_2^2 + x_3^2}{x_4}$$

The aggregate supply of the scale claim is normalized to one. The firm follows the marginal-pricing principle, represented in GEMB by the stationarity conditions with `behavior=:stationary`.

3. The consumer has utility function $u = c_1^{0.5} c_2^{0.25} c_3^{0.25}$, where c_1 , c_2 , and c_3 denote consumption of the product, labor, and land, respectively. The consumer is endowed with 100 units of labor, 100 units of land, and one unit of the scale claim.

The Pareto-optimal allocation of the original commodities corresponds to a maximizer of

$$F(x_2, x_3) = (x_2^2 + x_3^2)^{0.5} (100 - x_2)^{0.25} (100 - x_3)^{0.25}, \quad 0 \leq x_2, x_3 \leq 100$$

This function has three stationary points. Checking the boundary shows that the global maximum is attained at

$$x_2 = x_3 = \frac{200}{3}$$

while the other two stationary points are saddle points.

Let the price vector be $\mathbf{p} = (1, p_2, p_3, p_4)^\top$. The marginal products of the degree-one homogenized production function are

$$g_{x_2} = \frac{2x_2}{x_4}, \quad g_{x_3} = \frac{2x_3}{x_4}, \quad g_{x_4} = -\frac{x_2^2 + x_3^2}{x_4^2}$$

Since the aggregate supply of the scale claim equals one, market clearing implies $x_4 = 1$. The firm's stationarity conditions therefore give

$$p_2 = 2x_2, \quad p_3 = 2x_3, \quad p_4 = -(x_2^2 + x_3^2)$$

and hence

$$x_2 = \frac{p_2}{2}, \quad x_3 = \frac{p_3}{2}, \quad y = \frac{p_2^2 + p_3^2}{4}, \quad p_4 = -\frac{p_2^2 + p_3^2}{4}$$

Let the consumer's income be

$$w = 100p_2 + 100p_3 + p_4$$

The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, w) = \begin{bmatrix} 0 & \frac{w}{2} \\ \frac{p_2}{2} & \frac{w}{4p_2} \\ \frac{p_3}{2} & \frac{w}{4p_3} \\ 1 & 0 \end{bmatrix}$$

and

$$\mathbf{S}(\mathbf{p}) = \begin{bmatrix} \frac{p_2^2 + p_3^2}{4} & 0 \\ 0 & 100 \\ 0 & 100 \\ 0 & 1 \end{bmatrix}$$

Solving the corresponding equality-form wide-sense structural equilibrium model yields three equilibria, corresponding to the three stationary points of $F(x_2, x_3)$.

1. The first equilibrium, corresponding to the Pareto-optimal allocation, is

$$\mathbf{p}^* = \left(1, \frac{400}{3}, \frac{400}{3}, -\frac{80000}{9}\right)^\top, \quad w^* = \frac{160000}{9}$$

with

$$x_2^* = x_3^* = \frac{200}{3}$$

2. The second equilibrium is

$$\mathbf{p}^* = \left(1, 100 - \frac{100\sqrt{3}}{3}, 100 + \frac{100\sqrt{3}}{3}, -\frac{20000}{3} \right)^\top, \quad w^* = \frac{40000}{3}$$

with

$$(x_2^*, x_3^*) = \left(50 - \frac{50\sqrt{3}}{3}, 50 + \frac{50\sqrt{3}}{3} \right)$$

3. The third equilibrium is

$$\mathbf{p}^* = \left(1, 100 + \frac{100\sqrt{3}}{3}, 100 - \frac{100\sqrt{3}}{3}, -\frac{20000}{3} \right)^\top, \quad w^* = \frac{40000}{3}$$

with

$$(x_2^*, x_3^*) = \left(50 + \frac{50\sqrt{3}}{3}, 50 - \frac{50\sqrt{3}}{3} \right)$$

Hence degree-one homogenization yields three marginal-pricing equilibria corresponding to the three stationary points of the original allocation problem. Only the first corresponds to the Pareto-optimal allocation, while the other two correspond to saddle points.

The three equilibria can be obtained numerically from different starting values. The corresponding GEMB implementation is given in Listing [5.7](#).

Listing 5.7 Degree-One Homogenization of a Square-Sum Production Function with Multiple Equilibria

```
using GeneralEquilibriumModeling

beta = [0.5, 0.25, 0.25]
omega = 100.0

# Degree-one homogenization:
#  $g(x_1, x_2, x_4) = (x_1^2 + x_2^2) / x_4$ .
production_function = inputs -> begin
    x1, x2, x4 = inputs
     $(x_1^2 + x_2^2) / x_4$ 
end

# Scaled marginal product function:
# the common strictly positive factor  $1 / x_4$  is omitted.
marginal_product_function = inputs -> begin
    x1, x2, x4 = inputs
    [2.0 * x1, 2.0 * x2,  $-(x_1^2 + x_2^2) / x_4$ ]
end

cases = [
    ([60.0, 60.0], [1.0, 100.0, 100.0, -5000.0]),
    ([30.0, 70.0], [1.0, 60.0, 140.0, -5000.0]),
    ([70.0, 30.0], [1.0, 140.0, 60.0, -5000.0]),
]

function solve_case(case_id, input_start, p0)
    x1_start, x2_start = input_start

    model = GEMB.GEMBModel(
        [
            :product,
            GEMB.CommoditySpec(:labor; price_lower_bound=1.0e-10),
            GEMB.CommoditySpec(:land; price_lower_bound=1.0e-10),
            GEMB.CommoditySpec(
                :scale_claim;
                price_lower_bound=-Inf,
                price_upper_bound=Inf,
            ),
        ],
        numeraire=:product,
        numeraire_value=1.0,
    )

    GEMB.add_agent!(
        model,
        GEMB.ProductionFunctionSpec(
            production_function,
            marginal_product_function;
            behavior=:stationary,
        );
```


6 Intertemporal Equilibrium

6.1 Intertemporal Model Construction

The GEMB high-level framework does not use a separate model type for intertemporal equilibrium. A finite-horizon intertemporal economy is constructed as an ordinary `GEMBModel` in which time is represented by the `period` axis of dated commodities. Thus, the same behavioral specifications and equilibrium solver used for static models can also be used for intertemporal models.

Dated commodities are defined with `CommoditySpec`. For example,

```
product = GEMB.CommoditySpec(  
    :product;  
    axes=(period=1:4,),  
)
```

defines four dated product commodities. `CommodityRef` selects particular dated commodities or an entire intertemporal commodity path. For example, `CommodityRef(:product; period=2)` selects the period-2 product, whereas `CommodityRef(:product)` selects the product in all periods. A dated commodity can therefore be treated in exactly the same way as an ordinary commodity once its period has been specified.

A concrete economic agent is added with the ordinary `GEMB.add_agent!` interface. Its identity may contain a period through `AgentRef`, while `RelativePeriod` can be used to express commodity references relative to that period. In particular,

```
GEMB.RelativePeriod(0)  
GEMB.RelativePeriod(1)  
GEMB.RelativePeriod(-1)
```

denote the current, next, and previous periods, respectively. Hence a producer operating in period t may demand `CommodityRef(:product; period=RelativePeriod(0))` and supply `CommodityRef(:product; period=RelativePeriod(1))`, without explicitly writing the absolute period in each instance.

When the same economic structure is repeated over several periods, `AgentTemplate` provides a more convenient representation. For example,

```

producer = GEMB.AgentTemplate(
    :producer,
    spec;
    periods=1:3,
    demands=[
        GEMB.CommodityRef(
            :labor;
            period=GEMB.RelativePeriod(0),
        ),
    ],
    outputs=[
        GEMB.CommodityRef(
            :product;
            period=GEMB.RelativePeriod(1),
        ),
    ],
)

GEMB.add_agents!(model, producer)

```

creates the corresponding producers in periods 1, 2, and 3. `AgentTemplate` therefore describes a repeated structural pattern rather than a new type of economic behavior: `spec` remains an ordinary GEMB behavioral specification such as `CESSpec`, `CESMarginalUtilitySpec`, `MarshallDemandConsumerSpec`, or `ProductionFunctionSpec`.

Parameters that differ across the repeated periods can be specified with `ByPeriod`. For example,

```
claim_rate=GEMB.ByPeriod([0.10, 0.15, 0.20])
```

assigns the three values to the corresponding template periods. A scalar value, by contrast, is used unchanged in every period. `RelativePeriod` and `ByPeriod` thus serve different purposes: the former changes the dated object referred to by an agent, whereas the latter changes a parameter value across repeated agents.

Intertemporal claims, money, bonds, capital, and other dated objects do not require special commodity types. They can be represented as ordinary dated commodities and connected to economic agents through the usual GEMB demand, supply, endowment, claim, and auxiliary-condition mechanisms. Cross-period equilibrium relationships that cannot be represented directly by standard behavioral specifications can also be introduced through condition agents and observed equilibrium variables.

The basic construction procedure is therefore the same as for a static GEMB model: define dated commodities with `CommoditySpec`, create a `GEMBModel`, add concrete agents with `add_agent!`, expand repeated structures with `AgentTemplate` and `add_agents!` when necessary, and finally solve the resulting model with the ordinary GEMB equilibrium solver. The intertemporal tools mainly provide a convenient way to organize dated commodities, repeated agents, relative-period references, and period-varying parameters, while leaving the underlying equilibrium framework unchanged.

6.2 A Finite-Horizon Intertemporal Equilibrium Model

Consider a finite-horizon intertemporal equilibrium model in a closed economy. Assume the following:

1. The economy contains two types of commodities, a product and labor, and two types of economic agents, a firm and a consumer.
2. The consumer lives for n_p periods, and the economy also lasts for n_p periods.
3. The economy is closed and has no external borrowing. The initial supply of the product is exogenously given.
4. In period 1, the consumer is endowed with 150 units of the product. From period 1 through period $n_p - 1$, the consumer is endowed with 100 units of labor in each period.
5. The consumer consumes only the product and has demand for the product in periods $1, \dots, n_p$. Intertemporal preferences are represented by a single-level Cobb–Douglas utility function with equal share coefficients across periods.
6. There is one firm in each period from period 1 through period $n_p - 1$. In period t , the firm uses the product and labor as inputs to produce the product available in period $t + 1$. Its production function is

$$f(x, l) = 2\sqrt{xl}$$

The product used as an input is completely consumed in production.

The corresponding GEMB implementation is given in Listing [6.1](#).

Listing 6.1 A Finite-Horizon Intertemporal Equilibrium Model

```
# GEMB high-level framework: intertemporal equilibrium.
using GeneralEquilibriumModeling

np = 5

initial_product_endowment = 150.0
labor_endowment = 100.0
positive_price_floor = 1.0e-10

product = GEMB.CommoditySpec(
    :product;
    axes=(period=1:np,),
    price_lower_bound=positive_price_floor,
)

labor = GEMB.CommoditySpec(
    :labor;
    axes=(period=1:(np - 1),),
    price_lower_bound=positive_price_floor,
)

model = GEMB.GEMBModel(
    [
        product,
        labor,
    ];
    numeraire=GEMB.CommodityRef(
        :product;
        period=1,
    ),
    numeraire_value=1.0,
)

producer = GEMB.AgentTemplate(
    :producer,
    GEMB.CESSpec(
        [0.5, 0.5];
        es=1.0,
        alpha=2.0,
    );
    periods=1:(np - 1),
    demands=[
        GEMB.CommodityRef(
            :product;
            period=GEMB.RelativePeriod(0),
        ),
        GEMB.CommodityRef(
            :labor;
            period=GEMB.RelativePeriod(0),
        ),
    ],
)
```

6.3 Endogenous Interest Rates: A Four-Period 8×2 Pure-Exchange Model

6.3.1 Endogenous Interest Rates

In a structural equilibrium model, money can be treated as an **ad valorem claim**, with the interest rate interpreted as the corresponding **ad valorem claim rate** (Li, 2019, Chapter 7). When an economic agent borrows money to finance an expenditure of value V , an interest rate r generates an interest payment proportional to that value. In this sense, the monetary claim introduces a value wedge in the same way as other ad valorem claims.

In structural equilibrium models, the price of money in one period refers to the interest generated by one physical unit of money over that period. Since the choice of the physical unit of money is arbitrary, this price depends on the chosen unit of measurement. The capital value of one physical unit of money is the present value of the stream of future interest payments generated by that unit of money.

In structural equilibrium models, interest rates are often treated as exogenous parameters, for example, as rates determined by the central bank. An economy may involve multiple interest rates: rates may differ across periods, countries, financial markets, or economic agents. Given these rates, the equilibrium conditions determine commodity prices, activity levels, utility levels, and other endogenous variables. If one or more interest rates are instead treated as endogenous variables, the equilibrium system generally requires one additional independent auxiliary condition for each endogenous interest rate. Such auxiliary conditions may take the form of monetary policy rules, asset-pricing or no-arbitrage conditions, or intertemporal relationships between the value of money and the interest it generates.

An auxiliary condition used here is the **interest rate–interest relationship**. Let $v_{\text{int}}^{(t)}$ denote the present value of the interest generated in period t . Under a competitive money market and a constant money stock, the value of money at the beginning of period t equals the present value of the stream of interest generated from period t onward. Hence, the value of money remaining after the interest of period t has been received is $\sum_{i>t} v_{\text{int}}^{(i)}$. The endogenous interest rate therefore satisfies

$$r^{(t)} = \frac{v_{\text{int}}^{(t)}}{\sum_{i>t} v_{\text{int}}^{(i)}}$$

This interest rate–interest relationship can be added to the structural equilibrium conditions as an auxiliary condition, thereby allowing the equilibrium interest rate to be determined endogenously rather than specified in advance.

6.3.2 A Four-Period 8×2 Pure-Exchange Model

Consider a four-period pure-exchange economy. Interest rates are first treated as exogenous parameters. The interest rate–interest relationship is then introduced as an auxiliary condition to determine the interest rates endogenously. The velocity of money is assumed to be one in every period.

Assume the following:

1. The economy contains eight commodities: labor i and money i , $i = 1, 2, 3, 4$. There are two consumers, a laborer and a money owner (rentier). Labor 1 is taken as the numeraire commodity.
2. The laborer is endowed with $\omega_i > 0$ units of labor i , $i = 1, \dots, 4$, and the money owner is endowed with one physical unit of money in each period. Both consumers have the utility function $\prod_{i=1}^4 x_i^{\beta_i}$, where x_i denotes consumption of labor i , $\sum_{i=1}^4 \beta_i = 1$, and $\beta_1 > \beta_2 > \beta_3 > \beta_4 > 0$.
3. Let $r_i > 0$ denote the interest rate in period i . A purchase of labor i with value v requires an interest payment of $r_i v$.

Let $\mathbf{p} = (1, p_2, \dots, p_8)^\top$ and $\mathbf{w} = (w_1, w_2)^\top$, where w_1 and w_2 are the incomes of the two consumers. The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, \mathbf{w}) = \begin{bmatrix} \frac{\beta_1 w_1}{1+r_1} & \frac{\beta_1 w_2}{1+r_1} \\ \frac{\beta_2 w_1}{p_2(1+r_2)} & \frac{\beta_2 w_2}{p_2(1+r_2)} \\ \frac{\beta_3 w_1}{p_3(1+r_3)} & \frac{\beta_3 w_2}{p_3(1+r_3)} \\ \frac{\beta_4 w_1}{p_4(1+r_4)} & \frac{\beta_4 w_2}{p_4(1+r_4)} \\ \frac{r_1 \beta_1 w_1}{(1+r_1)p_5} & \frac{r_1 \beta_1 w_2}{(1+r_1)p_5} \\ \frac{r_2 \beta_2 w_1}{(1+r_2)p_6} & \frac{r_2 \beta_2 w_2}{(1+r_2)p_6} \\ \frac{r_3 \beta_3 w_1}{(1+r_3)p_7} & \frac{r_3 \beta_3 w_2}{(1+r_3)p_7} \\ \frac{r_4 \beta_4 w_1}{(1+r_4)p_8} & \frac{r_4 \beta_4 w_2}{(1+r_4)p_8} \end{bmatrix}$$

and

$$\mathbf{S} = \begin{bmatrix} \omega_1 & 0 \\ \omega_2 & 0 \\ \omega_3 & 0 \\ \omega_4 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}$$

Solving the equality-form wide-sense structural equilibrium model for given interest rates gives

$$p_j^* = \frac{\beta_j(1+r_1)\omega_1}{\beta_1(1+r_j)\omega_j}, \quad j = 2, 3, 4$$

and

$$p_{4+i}^* = \frac{r_i\beta_i(1+r_1)\omega_1}{\beta_1(1+r_i)}, \quad i = 1, 2, 3, 4$$

where $p_5^* = r_1\omega_1$. The prices $p_5^*, \dots, p_8^*$ are the present values of the interest payments in the four periods. The equilibrium incomes are

$$w_1^* = \omega_1 + \frac{(1+r_1)\omega_1}{\beta_1} \sum_{j=2}^4 \frac{\beta_j}{1+r_j}$$

and

$$w_2^* = r_1\omega_1 + \frac{(1+r_1)\omega_1}{\beta_1} \sum_{i=2}^4 \frac{r_i\beta_i}{1+r_i}$$

The interest rates can now be endogenized by adding the interest rate–interest relationship. Under the terminal condition $r_4 = +\infty$, the auxiliary conditions are

$$r_1 = \frac{p_5^*}{p_6^* + p_7^* + p_8^*}, \quad r_2 = \frac{p_6^*}{p_7^* + p_8^*}, \quad r_3 = \frac{p_7^*}{p_8^*}$$

Under the terminal condition $r_4 = +\infty$, the ordinary price of labor 4 converges to zero, while its interest-inclusive price may remain finite. Define the terminal interest-inclusive price as

$$\tilde{p}_4 := \lim_{r_4 \rightarrow +\infty} (1+r_4)p_4$$

Since the entire endowment ω_4 of labor 4 is traded in equilibrium, the value of the corresponding interest payment satisfies

$$p_8 = r_4 p_4 \omega_4$$

Therefore,

$$\tilde{p}_4 = \lim_{r_4 \rightarrow +\infty} r_4 p_4 = \frac{p_8}{\omega_4}$$

Thus, although r_4 is infinite and $p_4 = 0$ under the terminal condition, the effective unit cost of purchasing labor 4 remains finite and is represented by the terminal interest-inclusive price $\tilde{p}_4$.

Substituting the conditional-equilibrium expressions for $p_5^*, \dots, p_8^*$ into these auxiliary conditions gives

$$r_1^* = \frac{\beta_1}{\beta_2} - 1, \quad r_2^* = \frac{\beta_2}{\beta_3} - 1, \quad r_3^* = \frac{\beta_3}{\beta_4} - 1$$

Thus, the model illustrates a general way to endogenize interest rates in structural equilibrium models: first solve the equilibrium conditional on exogenous interest rates, and then add one independent auxiliary condition for each interest rate to be determined endogenously.

For the numerical example, let

$$(\beta_1, \beta_2, \beta_3, \beta_4) = (0.4, 0.3, 0.2, 0.1)$$

and suppose that the laborer is endowed with 100 units of each labor commodity.

The numerical equilibrium gives

$$p_1 = 1, \quad p_2 = \frac{2}{3}, \quad p_3 = \frac{1}{3}, \quad p_4 = 0,$$

$$p_5 = p_6 = p_7 = p_8 = \frac{100}{3},$$

and

$$r_1 = \frac{1}{3}, \quad r_2 = \frac{1}{2}, \quad r_3 = 1$$

The corresponding GEMB implementation is given in Listing 6.2. The two consumers are represented by Marshall-demand specifications, while a virtual condition agent is used to determine the first three interest rates and impose the terminal condition.

Listing 6.2 A Four-Period 8×2 Pure-Exchange Model with Endogenous Interest Rates

```
# GEMB high-level framework: four-period pure-exchange equilibrium
# with endogenous interest rates.
using GeneralEquilibriumModeling

beta = [0.4, 0.3, 0.2, 0.1]
labor_endowment = fill(100.0, 4)
money_endowment = ones(4)

positive_price_floor = 1.0e-10

labor = GEMB.CommoditySpec(
    :labor;
    axes=(period=1:3,),
    price_lower_bound=positive_price_floor,
)

money = GEMB.CommoditySpec(
    :money;
    axes=(period=1:4,),
    price_lower_bound=positive_price_floor,
)

model = GEMB.GEMBModel(
    [
        labor,
        :labor_4,
        money,
    ];
    numeraire=GEMB.CommodityRef(
        :labor;
        period=1,
    ),
    numeraire_value=1.0,
)

interest_rate_spec = GEMB.ConditionAgentSpec(
    (variables, observed_values) -> begin
        r1, r2, r3, terminal_interest_inclusive_price = variables
        p_money1, p_money2, p_money3, p_money4 = observed_values

        return [
            r1 - p_money1 / (p_money2 + p_money3 + p_money4),
            r2 - p_money2 / (p_money3 + p_money4),
            r3 - p_money3 / p_money4,
            terminal_interest_inclusive_price121 - p_money4 / labor_endowment[4],
        ]
    end,
)

GEMB.add_agent!(
    model,
```

6.4 A Basic Overlapping-Generations Trading Model

6.4.1 The Model

Consider a modified version of the basic overlapping-generations (OLG) trading model of Samuelson (1958). Assume the following:

1. The economy lasts for n_p periods and contains n_p dated commodities and $n_p - 1$ consumers.
2. Consumer 1 has utility function $u_1 = x_1^{\beta_1} x_2^{\beta_2} x_3^{1-\beta_1-\beta_2}$, where $\beta_1, \beta_2 \geq 0$ and $\beta_1 + \beta_2 \leq 1$.
3. Consumer i , $i = 2, \dots, n_p - 2$, has utility function $u_i = (x_i x_{i+1} x_{i+2})^{1/3}$.
4. Consumer $n_p - 1$ has utility function $u_{n_p-1} = x_{n_p-1}^{\bar{\beta}_1} x_{n_p}^{\bar{\beta}_2} x_1^{1-\bar{\beta}_1-\bar{\beta}_2}$, where $\bar{\beta}_1, \bar{\beta}_2 \geq 0$ and $\bar{\beta}_1 + \bar{\beta}_2 \leq 1$. When $\bar{\beta}_1 + \bar{\beta}_2 = 1$, the model has a time-line structure; when $\bar{\beta}_1 + \bar{\beta}_2 < 1$, consumer $n_p - 1$ also demands the first-period commodity and the model has a time-circle structure.
5. Consumer i , $i = 1, \dots, n_p - 1$, is endowed with 100 units of commodity i and 100 units of commodity $i + 1$.

Consider the case $n_p = 6$ and restrict equilibrium prices to a geometric path,

$$\mathbf{p} = (1, q, q^2, q^3, q^4, q^5)^\top, \quad q > 0$$

Let $\mathbf{w} = (w_1, \dots, w_5)^\top$ denote consumer incomes. Since consumer i owns commodities i and $i + 1$,

$$w_i = 100(p_i + p_{i+1}), \quad i = 1, \dots, 5$$

The demand and supply matrices are

$$\mathbf{D}(\mathbf{p}, \mathbf{w}) = \begin{bmatrix} \beta_1 w_1 & 0 & 0 & 0 & (1 - \bar{\beta}_1 - \bar{\beta}_2) w_5 \\ \frac{\beta_2 w_1}{p_2} & \frac{w_2}{3p_2} & 0 & 0 & 0 \\ \frac{(1-\beta_1-\beta_2)w_1}{p_3} & \frac{w_2}{3p_3} & \frac{w_3}{3p_3} & 0 & 0 \\ 0 & \frac{w_2}{3p_4} & \frac{w_3}{3p_4} & \frac{w_4}{3p_4} & 0 \\ 0 & 0 & \frac{w_3}{3p_5} & \frac{w_4}{3p_5} & \frac{\bar{\beta}_1 w_5}{p_5} \\ 0 & 0 & 0 & \frac{w_4}{3p_6} & \frac{\bar{\beta}_2 w_5}{p_6} \end{bmatrix}$$

and

$$\mathbf{S} = \begin{bmatrix} 100 & 0 & 0 & 0 & 0 \\ 100 & 100 & 0 & 0 & 0 \\ 0 & 100 & 100 & 0 & 0 \\ 0 & 0 & 100 & 100 & 0 \\ 0 & 0 & 0 & 100 & 100 \\ 0 & 0 & 0 & 0 & 100 \end{bmatrix}$$

The four endpoint preference parameters β_1 , β_2 , $\bar{\beta}_1$, and $\bar{\beta}_2$ are treated as endogenous. The equality-form wide-sense structural equilibrium model consists of the revenue-expenditure balance conditions and commodity-market clearing conditions, together with the geometric price-path restriction. Two equilibria are obtained.

The first is a **time-circle equilibrium**:

$$q^* = 1, \quad \mathbf{p}^* = (1, 1, 1, 1, 1)^\top, \quad \mathbf{w}^* = (200, 200, 200, 200, 200)^\top$$

with

$$\beta_1^* = 0, \quad \beta_2^* = \frac{2}{3}, \quad 1 - \beta_1^* - \beta_2^* = \frac{1}{3}$$

and

$$\bar{\beta}_1^* = \frac{1}{3}, \quad \bar{\beta}_2^* = \frac{1}{6}, \quad 1 - \bar{\beta}_1^* - \bar{\beta}_2^* = \frac{1}{2}$$

The corresponding demand matrix is

$$\mathbf{D}^* = \begin{bmatrix} 0 & 0 & 0 & 0 & 100 \\ \frac{400}{3} & \frac{200}{3} & 0 & 0 & 0 \\ \frac{200}{3} & \frac{200}{3} & \frac{200}{3} & 0 & 0 \\ 0 & \frac{200}{3} & \frac{200}{3} & \frac{200}{3} & 0 \\ 0 & 0 & \frac{200}{3} & \frac{200}{3} & \frac{200}{3} \\ 0 & 0 & 0 & \frac{200}{3} & \frac{100}{3} \end{bmatrix}$$

Since consumer 5 spends a positive share of income on commodity 1, the last generation is linked back to the first period, producing a time-circle structure.

The second is a **time-line equilibrium**. Let

$$q^* = \frac{3 + \sqrt{13}}{2}$$

Then

$$\mathbf{p}^* = (1, q^*, q^{*2}, q^{*3}, q^{*4}, q^{*5})^\top$$

and

$$\mathbf{w}^* = 100(1 + q^*)(1, q^*, q^{*2}, q^{*3}, q^{*4})^\top$$

The endpoint preference parameters are

$$\beta_1^* = \frac{5 - \sqrt{13}}{6}, \quad \beta_2^* = \frac{-1 + \sqrt{13}}{6}, \quad 1 - \beta_1^* - \beta_2^* = \frac{1}{3}$$

and

$$\bar{\beta}_1^* = \frac{1}{3}, \quad \bar{\beta}_2^* = \frac{2}{3}, \quad 1 - \bar{\beta}_1^* - \bar{\beta}_2^* = 0$$

The corresponding demand matrix is

$$\mathbf{D}^* = \frac{100(1 + q^*)}{3} \begin{bmatrix} \frac{5 - \sqrt{13}}{2} & 0 & 0 & 0 & 0 \\ \frac{-1 + \sqrt{13}}{2q^*} & 1 & 0 & 0 & 0 \\ \frac{1}{q^{*2}} & \frac{1}{q^*} & 1 & 0 & 0 \\ 0 & \frac{1}{q^{*2}} & \frac{1}{q^*} & 1 & 0 \\ 0 & 0 & \frac{1}{q^{*2}} & \frac{1}{q^*} & 1 \\ 0 & 0 & 0 & \frac{1}{q^{*2}} & \frac{2}{q^*} \end{bmatrix}$$

In this equilibrium, $1 - \bar{\beta}_1^* - \bar{\beta}_2^* = 0$, so the last consumer has no demand for the first-period commodity and the intergenerational trading structure forms a time line rather than a circle.

Both solutions represent steady-state equilibrium paths. Although the finite-horizon model contains special endpoint consumers to close the six-period economy, its interior periods reproduce the same intergenerational structure under a one-period shift of calendar time and cohort index. In particular, the endowment and age-specific demand patterns are unchanged, while prices and incomes are multiplied by the constant factor q^* . Hence, abstracting from the endpoint closure, the interior equilibrium structure can be extended indefinitely forward and backward in time, yielding the corresponding steady-state OLG equilibrium path.

6.4.2 Computational Implementation

The model is implemented by treating products delivered in different periods as different commodities. For $n_p = 6$, the economy therefore contains six dated commodities with price vector $\mathbf{p} = (p_1, \dots, p_6)^\top$, and the first-period product is chosen as the numeraire, so that $p_1 = 1$.

Consumer i is endowed with 100 units of commodities i and $i + 1$. Hence its income is determined directly by prices as $w_i = 100(p_i + p_{i+1})$ and does not need to be introduced as a separate equilibrium variable. Consumers 2–4 have equal-share Cobb–Douglas preferences and are represented directly by `CESSpec` with three shares equal to $1/3$.

The preference shares of consumers 1 and 5 are endogenous. Their demands are therefore specified by `MarshallDemandConsumerSpec`. Consumer 1 observes the endogenous variables β_1 and β_2 and has expenditure shares β_1 , β_2 , and $1 - \beta_1 - \beta_2$. Consumer 5 similarly observes β_1 and β_2 and has expenditure shares β_1 , β_2 , and $1 - \beta_1 - \beta_2$.

A `ConditionAgentSpec` is introduced to determine the steady-state price path and the four endpoint preference parameters. Its endogenous variables are

$$(\log q, \beta_1, \beta_2, \bar{\beta}_1, \bar{\beta}_2)$$

where $q = \exp(\log q) > 0$. The five associated conditions impose the geometric price path

$$p_{t+1} = qp_t, \quad t = 1, \dots, 5$$

Together with $p_1 = 1$, these conditions imply

$$\mathbf{p} = (1, q, q^2, q^3, q^4, q^5)^\top$$

Although the four preference parameters do not appear explicitly in these five price-path conditions, they enter the demands of consumers 1 and 5 and therefore affect commodity-market clearing. `PATH` consequently solves prices, q , and the four preference parameters jointly so that both the geometric price conditions and all market-clearing conditions are satisfied.

After fixing the numeraire, there are five ordinary price variables $p_2, \dots, p_6$ and five additional endogenous variables in the condition agent. Correspondingly, Walras' law leaves five independent commodity-market clearing conditions, while the condition agent supplies five price-path equations. The resulting system is therefore closed.

The model has multiple equilibria. To obtain the two equilibria reported above, the same initial values $(0.1, 0.1, 0.1, 0.1)$ are used for the four preference parameters, while only the

initial value of q is changed. The time-circle equilibrium is obtained from $q_0 = 0.5$, whereas the time-line equilibrium is obtained from $q_0 = 2$.

The corresponding GEMB implementation is given in Listing [6.3](#).

Listing 6.3 A Basic Overlapping-Generations Trading Model

```
using GeneralEquilibriumModeling

const GEM = GeneralEquilibriumModeling.GEM
const GEMB = GeneralEquilibriumModeling.GEMB

np = 6
omega = 100.0
product(t) = GEMB.CommodityRef(:product; period=t)

function build_model(q0, beta0)
    model = GEMB.GEMBModel(
        [
            GEMB.CommoditySpec(
                :product;
                axes=(period=1:np,),
                price_lower_bound=1.0e-10,
            ),
        ];
        numeraire=product(1),
    )

    # q and endpoint preference shares
    GEMB.add_agent!(
        model,
        GEMB.ConditionAgentSpec(
            (v, p) -> begin
                q = exp(v[1])
                [p[t + 1] - q * p[t] for t in 1:(np - 1)]
            end,
        );
        variable_names=[:log_q, :beta1, :beta2, :beta_bar1, :beta_bar2],
        variable_start=[log(q0); beta0],
        variable_lower_bounds=-Inf,
        variable_upper_bounds=Inf,
        observed_variables=[
            GEM.PriceVariableRef(Symbol(:product_, t))
            for t in 1:np
        ],
        name=:steady_state,
    )

    # Consumer 1
    GEMB.add_agent!(
        model,
        GEMB.MarshallDemandConsumerSpec(
            (w, p, b) -> [
                b[1] * w / p[1],
                b[2] * w / p[2],
                (1 - b[1] - b[2]) * w / p[3],
            ],
        );
```

References

- Björk, T., A. Murgoci and X. Y. Zhou (2014) Mean–Variance Portfolio Optimization with State-Dependent Risk Aversion. *Mathematical Finance*, 24(1): 1-24.
- Bonnisseau, J. M., and Cornet, B. (1990) Existence of Marginal Cost Pricing Equilibria in Economies with Several Nonconvex Firms. *Econometrica*, 58(3): 661-682.
- Ferris, Michael C., and Todd S. Munson (2000) Complementarity Problems in GAMS and the PATH Solver. *Journal of Economic Dynamics and Control*, 24(2): 165-188.
- Fullerton, D. (1989) Notes on Displaced CES Functional Forms. Unpublished manuscript.
- Kemeny, J. G., O. Morgenstern and G. L. Thompson (1956) A Generalization of the von Neumann Model of an Expanding Economy. *Econometrica*, 24(2): 115-135.
- LI Wu (2019) General Equilibrium and Structural Dynamics: Perspectives of New Structural Economics. Beijing: Economic Science Press. (In Chinese)
- Mas-Colell, A., Whinston, M. D., Green, J. R. (1995) *Microeconomic Theory*. New York: Oxford University Press.
- McKenzie, L. W. (2002) *Classical General Equilibrium Theory*. MIT Press.
- Nakamura, Yutaka (2015) Mean-Variance Utility. *Journal of Economic Theory*, 160: 536-556.
- Pollak, Robert A. (1977) Price Dependent Preferences. *American Economic Review*, 67(2): 64–75.
- Quinzii, M. (1992) *Increasing Returns and Efficiency*. Oxford University Press.
- Samuelson, P. A. (1958) An Exact Consumption-Loan Model of Interest with or without the Social Contrivance of Money. *Journal of Political Economy*, 66(6): 467-482.
- Torres, Jose L. (2016) *Introduction to Dynamic Macroeconomic General Equilibrium Models (Second Edition)*. Vernon Press.
- von Neumann, J. (1945) A Model of General Economic Equilibrium. *The Review of Economic Studies*, 13(1): 1-9. English translation of von Neumann (1937).